

Final Project

RV64I RISC-V Execution Unit

ENSC 350 – Digital System Design

Due: April 12th, 2026

By: Kevin Shen

Table of Contents

Table of Contents	1
Objective	2
1. Subsystem Design Process ArithUnit	2
1.1 Baseline Design	3
1.2 Conditional Sum Adder Design	3
1.3 Block Carry Look-Ahead Adder Design	4
1.4 Carry-Skip Adder Design	5
1.5 Carry Select Adder Design	7
1.6 Post-Fit Results	7
2. Subsystem Design Process ShiftUnit	8
2.1 BarrelShifter Design Techniques	9
2.2 Shift Subsystem Variations	12
2.3 Top 3 ShiftUnit Topologies	15
3. Subsystem Design Process LogicUnit	16
4. Design Process ExecUnit	17
2. Testbenches and Test Vectors	19
2.1 Testbench Design	20
2.2 Test Vectors	20
3. Results & Analysis	21
3.1 Waveform Analysis	22
3.2 Predictions	25
3.2.1 LEs and ALMs	25
3.2.1.1 ArithUnit	25
3.2.1.2 ShiftUnit	28
3.2.1.3 LogicUnit	29
3.2.1.3 ExecUnit	29
3.2.2 Timing Delays	30
3.2.2.1 ArithUnit	30
3.2.2.1 ShiftUnit	33
3.2.2.1 ExecUnit	34
3.3 Cost Analysis	35
3.4 Timing Analysis	36
3.5 Interpreting Results	37
3.5.1 Resource Usage Predicted vs Measured	37
3.5.2 Timing Delay Predicted vs Measured	38
3.5.2 Results	39
3.6 Design Candidate Considerations	40
3.7 Optimized Cost-Performance Design Candidate	41
Appendix	49

Objective

The objective of this report is to design and simulate multiple 64-bit adder architectures using testbenches for verification. The two topologies evaluated are the Ripple-Carry and Conditional-Sum adders. Both designs were implemented on Cyclone IV and Arria II FPGAs, producing four candidates for cost and resource comparison.

1. Subsystem Design Process ArithUnit

For the arithmetic subsystem (ArithUnit.vhd), the adder topologies from Adder.vhd are reused, including the baseline ripple-carry, Conditional-Sum, Block Carry Look-Ahead, Carry-Skip, and Carry-Select architectures. Each topology is bound to the ArithUnit wrapper via VHDL configurations, allowing the adder architecture to be swapped without modifying the ArithUnit source code.

AddnSub implements two's complement subtraction by inverting all bits of B and forcing Cin = '1' when AddnSub = '1', computing $A + (\sim B) + 1 = A - B$. When AddnSub = '0', B and Cin pass through unchanged for addition.

ExtWord supports 32-bit instructions by zeroing the upper 32 bits of both A and B before addition, preventing the upper bits from affecting the lower 32-bit result. The upper 32 bits of the output are then sign-extended from bit 31, matching RISC-V W-type instruction behaviour. Note that Cout and Ovfl are not valid for 32-bit operations per the FP specification.

Port	Width	Direction	Description
A	64	in	Operand A
B	64	in	Operand B
Cin	1	in	Carry in
AddnSub	1	in	'0' = add, '1' = subtract
ExtWord	1	in	'1' for 32-bit mode
S	64	out	Sum or difference
Cout	1	out	Carry out, valid for 64-bit only
Ovfl	1	out	Overflow, valid for 64-bit only

1.1 Baseline Design

For the baseline design, full adders are instantiated using generate for-loop for the equations

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + C_i (A_i \oplus B_i)$$

Where i is a 64-bit vector, $C_0 = C_{in}$ and $C_{out} = C_{64}$. Signed overflow is detected by

$$Ovfl = S_N \oplus S_{N-1}$$

$$S_N \oplus S_{N-1} = (P_N \oplus C_N) \oplus (P_{N-1} \oplus C_{N-1}) = (P_{N-1} \oplus C_N) \oplus (P_{N-1} \oplus C_{N-1})$$

$$Ovfl = (P_{N-1} \oplus P_{N-1}) \oplus (C_N \oplus C_{N-1}) = C_N \oplus C_{N-1}$$

Where P_N (Propagate) becomes P_{N-1} due to sign extension.

A structural ripple-carry adder was implemented using a generate for-loop because of the scalability, allowing N number of full adders for 2^N number of bits. Signed overflow is detected by S_{64} xor S_{63} but can be simplified to be C_{64} xor C_{63}

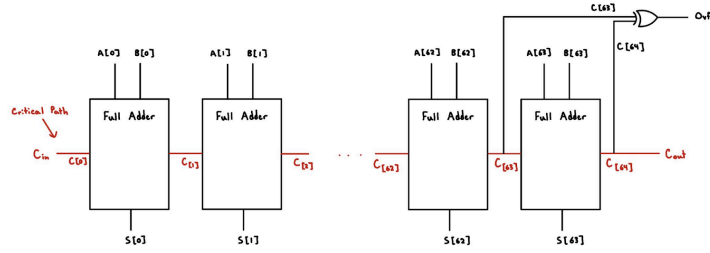


Figure 1. Circuit Diagram of structural baseline Ripple-Carry Adder

1.2 Conditional Sum Adder Design

The conditional-sum adder utilizes the carry-select principle, which computes two possible sums in parallel for each block: one assuming a carry-in of 0 and the other assuming a carry-in of 1. Once the actual carry-in from the preceding block is determined, a multiplexer selects the correct precalculated result, reducing overall propagation delay compared to a ripple-carry adder.

This topology was implemented recursively by establishing a base case for $N = 1$ that instantiates the Baseline entity. For $N > 1$, a generate statement instantiates three sub-instances of the CarrySelect entity: the first calculates the lower $N/2$ sum bits using the input carry, while the second and third calculate the upper $N/2$ sum bits assuming carry-in values of 0 and 1, respectively. The carry-out from the first instance then selects the correct upper $N/2$ sum bits and final carry-out from the second and third instances via conditional assignments. Overflow detection uses the same sign-bit analysis method described in the previous section.

$$Ovfl = \overline{(A_{N-1} \oplus B_{N-1})} \cdot (S_{N-1} \oplus A_{N-1})$$

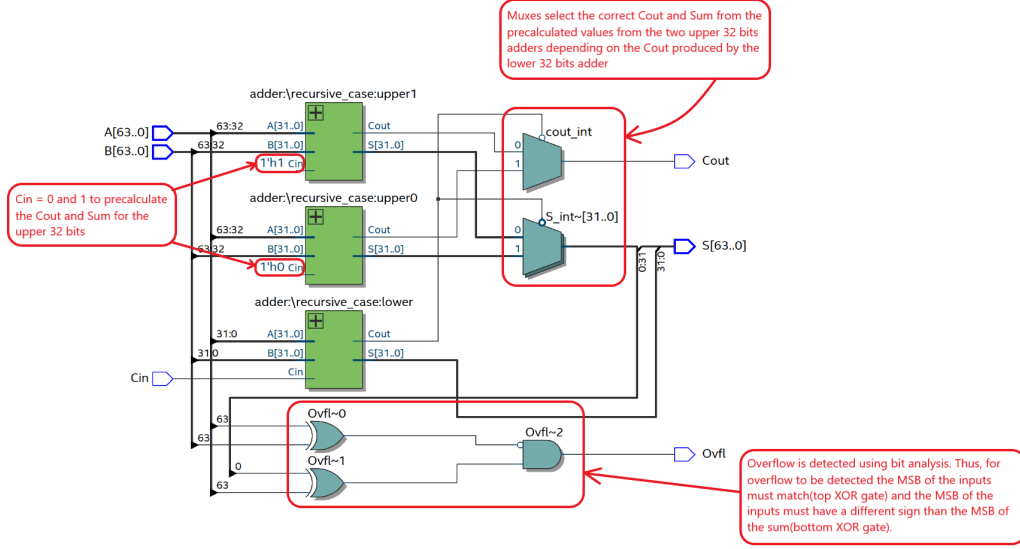


Figure 2. Circuit Diagram of Conditional Sum Adder(with 3 instances of 32-bit CSAs)

1.3 Block Carry Look-Ahead Adder Design

The block carry look-ahead architecture utilizes the carry look-ahead principle, which calculates all intermediate carry signals simultaneously based on generate and propagate logic using the formula:

$$C_{i+1} = G_i + P_i C_i,$$

Where Propagate and Generate are

$$P_i = A_i \oplus B_i \text{ and } G_i = A_i B_i$$

In block carry look-ahead the carry look-ahead formula is split into multiple blocks where the carries in each block start calculating at the same time, but must wait for the previous block's C_{OUT} in order to start.

This topology was implemented in VHDL by using a generate loop to instantiate 16 blocks where each block is a 4 bit carry look-ahead adder associated with 0-3, 4-7, 8-11, ..., 56-59, 60-63 bits that create their respective internal carries using carry look-ahead where C_0 is C_{in}

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + G_0 P_1 + P_1 P_0 C_0 \\ C_3 &= G_2 + G_1 P_2 + G_0 P_2 P_1 + P_2 P_1 P_0 C_0 \end{aligned}$$

The sum is then computed by

$$S_i = P_i \oplus C_i$$

Each 4-bit block produces a block propagate and block generate

$$P_{block,k} = P_3 P_2 P_1 P_0$$

$$G_{block,k} = G_3 + G_2 P_3 + G_1 P_3 P_2 + G_0 P_3 P_2 P_1$$

A generate loop is then used to compute the block-level carries for k from 3 to 15 using:

$$C_{k+1} = G_{Block,k} + P_{Block,k} C_k$$

In this implementation blocks 1 to 3 can be calculated by expanding using full look-ahead and the remaining block carries are calculated recursively

Cout is the carry out from block 16 and ovfl is calculated in a similar way as the baseline design, xor with the last 2 carries, Cout from block 15 xor Cout from block 16 as in this topology the carry vector C represents block-level carries rather than bit-level carries as internal carries are not explicitly stored. This makes C(15) xor C(16) equivalent to C_63 xor C_64 when calculating overflow.

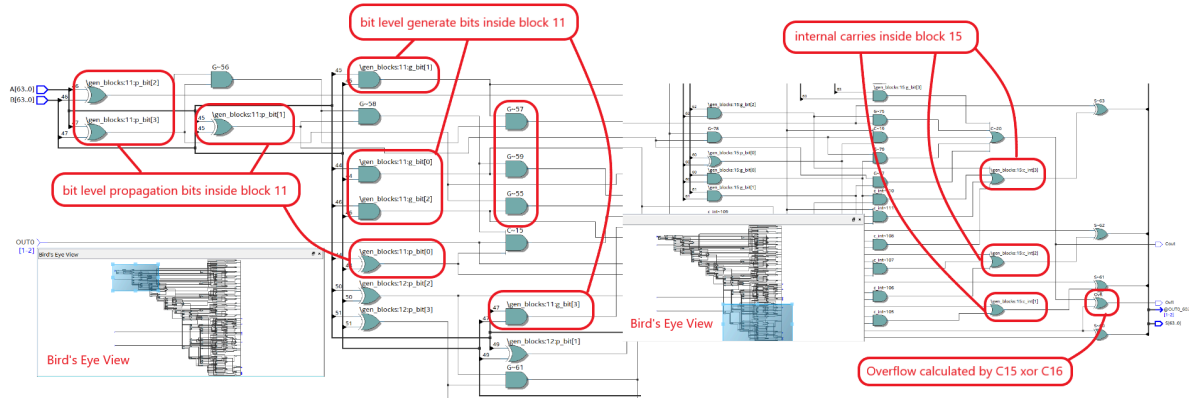


Figure 3. Circuit Diagram of Block Carry Look-Ahead adder with 16 blocks

1.4 Carry-Skip Adder Design

The purpose of this design is to implement a 64-bit carry-skip adder that improves upon the delay limitations of a ripple-carry adder by allowing carry signals to bypass blocks of bits when possible.

The 64-bit adder is partitioned into 4-bit blocks, resulting in a total of 16 blocks. Each block internally uses a ripple-carry structure, with additional logic to enable carry skipping.

For each bit, a propagate and generate signal is defined as:

$$P_i = A_i \oplus B_i, G_i = A_i \cdot B_i$$

The propagate signals are combined using a reduction AND operation to form the block propagate signal:

$$P_{block} = P_0 \cdot P_1 \cdot P_2 \cdots P_{i-1}$$

This signal determines whether the carry-in can bypass the entire block. The carry-out of each block is computed as:

$$C_{out} = P_{block} \cdot C_{in} + \overline{P_{block}} \cdot C_{ripple}$$

Where C_{ripple} is the carry produced by the internal ripple chain. If an $P_{block} = 1$, the carry skips the block. Otherwise, it propagates through the ripple path.

This structure introduces two possible carry paths: a ripple path (bit-by-bit propagation) and a skip path (block-level bypass), reducing the overall worst-case delay compared to a purely ripple-carry design.

Overflow detection follows the same method described in Section 1.1 using sign-bit analysis.

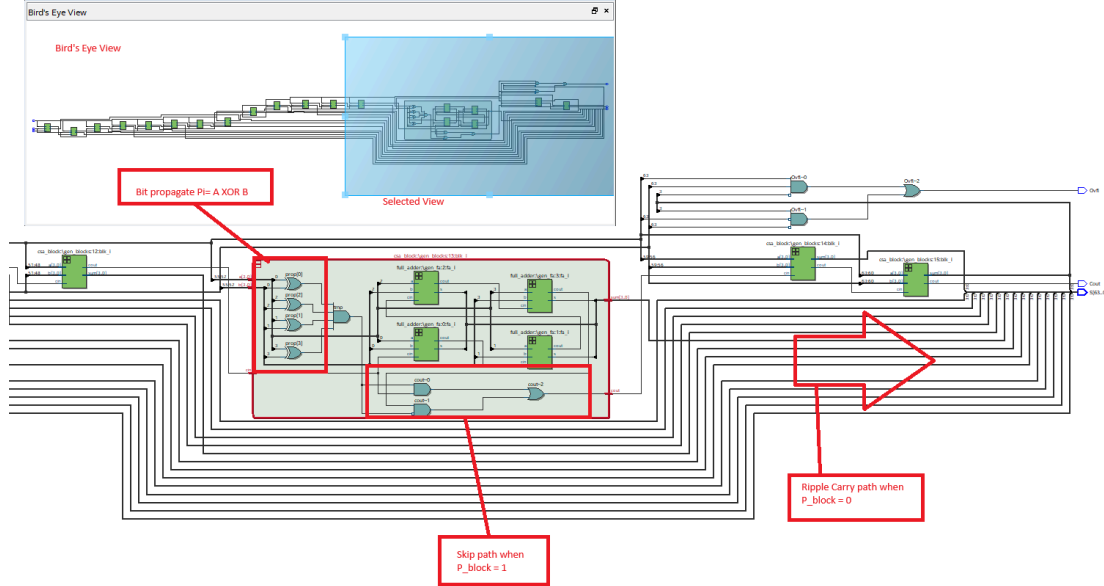


Figure 4: RTL implementation of the 64-bit carry skip adder, highlighting block-based ripple structure and skip logic that allows carry to bypass blocks when $P_{block}=1$.

1.5 Carry Select Adder Design

The Carry-Select adder implementation utilizes the carry-select principle described in Section 1.2 for the conditional-sum adder design.

The 64-bit carry select adder splits the input into 7 unequal blocks of 4, 4, 8, 8, 16, 16, and 8 bits. The blocks increase in size toward the MSB because each successive block has one additional mux delay before its carry arrives, giving it more time to complete its ripple.

The first block (bits 0:3) is a single instantiation of the Baseline ripple carry adder using the real carry-in. Every subsequent block is implemented as two parallel Baseline instantiations of the same bit width, one with Cin forced to '0' and one with Cin forced to '1', both computing their sum, carry-out.

Once the true carry arrives, it selects the pre-computed sum and carry-out from whichever instance assumed the correct carry-in, which is repeated for all subsequent blocks. Since overflow is only meaningful for the full 64-bit result, all sub-block overflow outputs are left open except for the last block (bits 56:63) and because overflow is determined in the Baseline architecture, no additional overflow logic is needed.

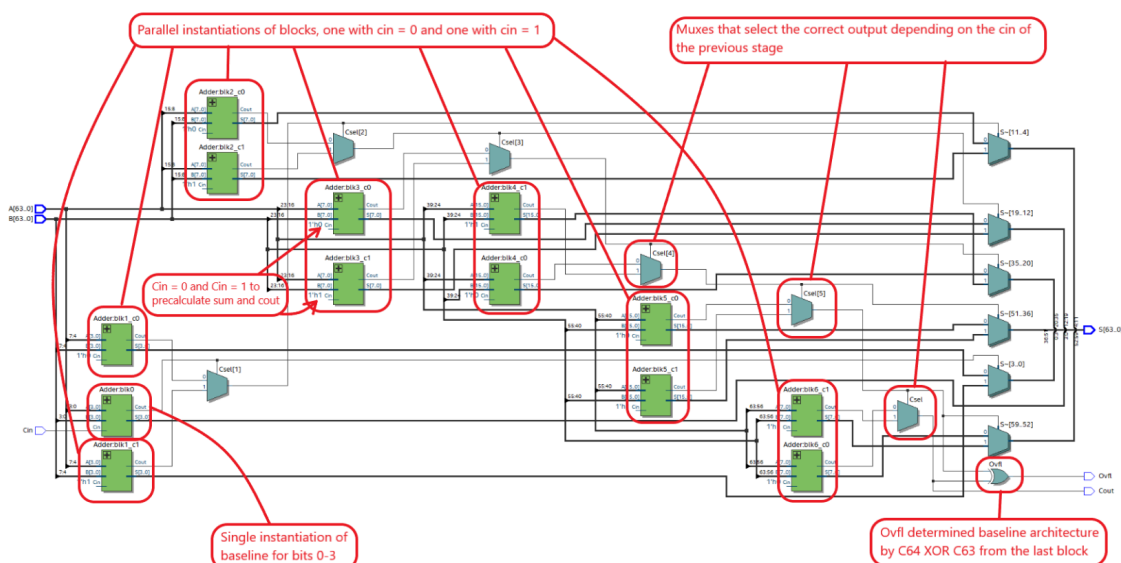


Figure 5. Circuit Diagram of CarrySelect Adder with 13 instances of Baseline Ripple Carry Adders

1.6 Post-Fit Results

The post-fit technology maps for each adder topology are provided in the Appendix and confirm the structural behaviour described in the design sections.

The baseline ripple adder (Figures A.1–A.2) shows the expected serial carry propagation structure, with each LOGIC_CELL_COMB forming a full-adder stage. On Arria II the dedicated carry chain is

visible connecting each stage, explaining why the FastRipple implementation achieves significantly better timing than the gate-level model predicts, as the synthesizer maps the carry chain onto hardened carry primitives rather than general-purpose LUT logic.

The conditional sum adder (Figures A.3–A.4) clearly shows the three-instance recursive structure, with the lower block generating carry_mid which selects between two precomputed upper-half results through a MUX selection stage. On Arria II the design collapses to 44 ALMs versus 291 LEs on Cyclone IV, visually confirming the aggressive logic minimization Quartus applies to the recursive structure.

The carry skip adder (Figures A.5–A.6) shows each 4-bit block containing a ripple-carry chain alongside the skip multiplexer, with the two carry paths. These two paths are ripple when P_block=0, and skip when P_block=1, which are both visible in the annotated diagrams.

The block carry look-ahead adder (Figures A.7–A.8) shows the 16×4 -bit block structure with the block-level carry net and generated loop visible. On Arria II the wider ALUTs absorb the fan-in of the generate and propagate terms into slightly fewer cells, consistent with the measured ALM count of 152 versus 273 LEs on Cyclone IV.

The carry select adder (Figures A.9–A.10) shows the parallel Cin=0 and Cin=1 baseline instances for each block, with the MUX selection logic and final carry-out and overflow outputs clearly annotated. The carry propagation path visible on Cyclone IV confirms that the critical path runs through the sequential MUX chain rather than the parallel ripple instances.

2. Subsystem Design Process ShiftUnit

Port Description

Port	Width	Mode	Description
A	64	In	Operand to shift
B	64	In	Shift amount source
ShiftFN	2	In	Shift operation select
ExtWord	2	In	‘1’ for 32-bit mode
Y	64	Out	Shift result

2.1 BarrelShifter Design Techniques

Optimizing the length of a barrel shifter

A barrel shifter breaks the shift amount into individual bits and dedicates one stage to a bit or a set of bits. For a 64-bit shifter with 6 stages, that means 6 shift amounts where stage i either shifts the data by 2^i positions or passes it through, controlled by bit i of the shift amount. The length optimization replaces this one-bit-per-stage scheme with wider MUX stages that consume multiple bits at once. For instance a 4-channel stage reads 2 bits simultaneously and can shift by 0, $1 \cdot \text{base}$, $2 \cdot \text{base}$, or $3 \cdot \text{base}$ in a single stage depending on the select signal, where the base for this instance is 4^i , reducing the stages from 6 to 3. This method can also be used to create barrel shifters of unequal shift amounts as long as the product of channel inputs matches the desired number of outcomes.

For example a 64-bit shifter with 3 stages can be implemented in VHDL by first declaring a type called Stage as an array of 4, 64-bit vectors and creating a signal for that type.

```
type Stage64 is array (0 to 3) of std_logic_vector(63 downto 0);
signal s64    : Stage64;
```

Then using a generate for loop indexed by i to iterate over each of the 3 stages, then nested a generate for loop indexed by j to iterate over all 64-bits in order to manipulate the array.

```
gen_stages : for i in 0 to 2 generate
  gen_64_bits : for j in 0 to 63 generate
```

The shift amount is determined by `base` and the corresponding select signal is determined by `sel` where `B` is the input of the shiftunit and is converted to an integer from unsigned. The new bit positions within the array are thus determined using `pos`

```
variable base : integer := 4**i;
sel := to_integer(unsigned(B(2*i+1 downto 2*i)));
pos := j + sel * base;
```

Then any manipulated j bits in the current iteration, i , are then stored into the next array `s64(i+1)` to be then processed by the next iteration until all stages are completed.

```
if B(i) = '1' then
  s64(i+1)(j) <= s64(i)(j + shift_amt);
else
  s64(i+1)(j) <= s64(i)(j);
end if;
else
  if B(i) = '1' then
    s64(i+1)(j) <= fill64;
  else
    s64(i+1)(j) <= s64(i)(j);
  end if;
```

Combining logical shift operations into a single barrel shifter

A basic shifter would require separate barrel shifters for left and right shifts because the data connections run in opposite directions. This can be overcome using two different methods. The first

method is bit-reversal, where the input bits are mirrored before entering the right-shift chain, the right-shift logic is applied as normal, and the output bits are then mirrored again to produce the correct left-shift result. This works because mirroring the bit order of a word turns a right shift into a left shift, allowing a single right-shift chain to handle both directions. The second method is the unified bidirectional chain, where a single chain handles both directions by incorporating a direction check inside every stage. Each stage reads from $j + \text{shift amount}$ for a right shift or $j - \text{shift amount}$ for a left shift where j is the current bit position, which works because when an element goes below 0 in an array the element becomes 0, producing the zero fill on the right side of a left shift without fill logic. For right shifts when the source index $j + \text{shift amount}$ exceeds 63 the fill bit is used depending on the instruction SRA/SRL.

The bit reversal method can be implemented in VHDL by first mirroring all the input bits A using a generate for loop if it is a left shift.

```
gen_rev_in : for i in 0 to N-1 generate
    A_rev(i) <= A_in(N-1-i) when is_sll = '1' else A_in(i);
end generate;
```

The fill bit is then determined based on the shift type: 0 for SLL and SRL, $A(63)$ for SRA, or $A(31)$ for SRAW, where $A(63)$ and $A(31)$ represent the most significant bits for SRA and SRAW, respectively.

```
fill <= A(31) when (ShiftFN = "11" and ExtWord = '1')
    else A(63) when (ShiftFN = "11" and ExtWord = '0')
    else '0';
```

Then precede through the right shift logic where the index i is the stage and j is the current bit position and B is the input from the shiftunit.

```
begin
    if j + shift_amt <= N-1 then
        if B(i) = '1' then
            stage(i+1)(j) <= stage(i)(j + shift_amt);
        else
            stage(i+1)(j) <= stage(i)(j);
        end if;
    else
        if B(i) = '1' then
            stage(i+1)(j) <= fill;
        else
            stage(i+1)(j) <= stage(i)(j);
        end if;
    end if;
end process;
```

Then the bits are mirrored back from the last stage using a generate for loop if it is a left shift producing the same results as a dedicated left barrel shifter.

```
gen_rev_out : for i in 0 to N-1 generate
    Y_rev(i) <= stage(6)(N-1-i) when is_sll = '1' else stage(6)(i);
end generate;
```

For the unified bidirectional chain VHDL implementation, a control signal is first assigned to determine whether a left-shift operation is performed. The fill logic follows the same approach as the bit-reversal method.

```
is_left <= '1' when ShiftFN = "01" else '0';
```

Then preceding into the shift logic there is an if statement to check if a left shift should occur. If it's a right shift then the new position of the j bits are calculated by $j + \text{shift_amt}$ but if it is a left shift then the new position will be calculated by $j - \text{shift_amt}$.

```
if is_left = '0' then
  if j + shift_amt <= 63 then
    if B(i) = '1' then
      s64(i+1)(j) <= s64(i)(j + shift_amt);
    else
      s64(i+1)(j) <= s64(i)(j);
    end if;
  else
    if B(i) = '1' then
      s64(i+1)(j) <= fill;
    else
      s64(i+1)(j) <= s64(i)(j);
    end if;
  end if;
else
  if j - shift_amt >= 0 then
    if B(i) = '1' then
      s64(i+1)(j) <= s64(i)(j - shift_amt);
    else
      s64(i+1)(j) <= s64(i)(j);
    end if;
  else
    if B(i) = '1' then
      s64(i+1)(j) <= '0';
    else
      s64(i+1)(j) <= s64(i)(j);
    end if;
  end if;
end if;
```

Combining 64-bit/32-bit operations into a single barrel shifter

A basic barrel shifter would require a separate 32-bit barrel in parallel with the 64-bit one, selecting between their outputs based on the ExtWord signal. This can be overcome using two methods. The first is zero-upper, where the upper 32 bits of the input are forced to zero, the shift amount is capped at 5 bits, and the lower 32 bits of the result are sign-extended upward from bit 31. The second is SwapWord, where the 32-bit operand is placed in the upper half of the 64-bit input before entering the chain, so the result lands in the upper 32 bits and sign-extension fills downward. SwapWord is particularly convenient for right shifts because the sign bit naturally sits at position 63 after the swap, acting as the fill bit for SRA without any explicit special-casing, whereas zero-upper requires the fill bit to be explicitly selected based on ExtWord. For both methods the shifter chain itself is completely unmodified, changing only the input and output processing.

For VHDL implementation for the zero-upper method the 5th bit is set to 0 when ExtWord is 1 and is set to B(5) when ExtWord is 0, where B is the input to the shiftunit. So the max shift is 32 for 32-bit shift instructions.

```
shift_amt(4 downto 0) <= B(4 downto 0);
```

```
shift_amt(5)          <= B(5) when ExtWord = '0' else '0';
```

Since no changes are made to the shifter chain the shifting logic precedes as normal. Then the upper 32 bits are all set to be the 31st bit if ExtWord is 1.

```
if ExtWord = '1' then
    Y(63 downto 32) <= (others => Y_int(31));
else
    Y(63 downto 32) <= Y_int(63 downto 32);
end if;
```

The swapword method also sets the 5th bit to 0 when ExtWord is 1 and is set to B(5) when ExtWord is 0. The only difference is the output processing where for right shifts the 32-bit answer is stored in the upper 32-bit and needs to be moved to the lower 32-bits and the upper 32-bit are then signextended with 63-bit when ExtWord = 1.

```
if ExtWord = '1' then
    Y_sr(31 downto 0) <= sr64(63 downto 32);
    Y_sr(63 downto 32) <= sr64(others => stage(6)(63));
else
    Y <= sr64(6);
end if;
```

For left shifts the 32-bit answer is already stored in the lower 32-bits so the upper 32-bit just need to sign extended with 31-bit when Extend = 1.

```
if ExtWord = '1' then
    Y_sll(31 downto 0) <= sll64(6)(31 downto 0);
    Y_sll(63 downto 32) <= (others => sll64(6)(31));
else
    Y_sll <= sll64(6);
end if;
```

2.2 Shift Subsystem Variations

Term	Description
Dual	Separate 64bit and 32bit barrel shifters
SingleSW/SingleZU	Method of combining 64 bit and 32 bit into a single barrel shifter (SwapWord or Zero Upper)
nCh	n-channel muxes used
Sep	Separate SRL/SRA/SLL barrel shifters
Uni/Rev	Method of combining right and left logical shifts into a single barrel shifter(Reversal or Unified bidirectional chain)

All variations are implemented on a Cyclone IV device to validate the technique, after which the three selected topologies will be evaluated on the Arria II.

Dual2ChUni

This architecture uses 6 stages of 2-1 channel muxes for 64-bit shifts and combines SRL/SRA/SLL into a single shifter using the unified bidirectional chain method. 32-bit shifts are handled the same way with a separate barrel shifter using 5 stages instead.

Dual4ChUni

Similar to Dual2ChUni architecture but this architecture uses 3 stages of 4-1 channel muxes for 64-bit shifts. 32-bit shifts are 2 stages of 4-1 muxes and 1 stage of a 2-1 mux.

Dual8ChUni

Similar to Dual2ChUni architecture but this architecture uses 2 stages of 8-1 channel muxes for 64-bit shifts. 32-bit shifts are 1 stage of 8-1 muxes and 1 stage of a 4-1 mux.

SingleZU2ChRev

This architecture uses 6 stages of 2-1 channel muxes but combines SLL/SRL/SRA to all use the same barrelshifter using the bit-reversal method to deal with SLL. 64-bit and 32-bit shifts are all done in the same barrel shifter using the zero-upper method.

SingleZU4ChRev

Similar to SingleZU2ChRev architecture but this architecture uses 3 stages of 4-1 channel muxes instead.

SingleZU8ChRev

Similar to SingleZU2ChRev architecture but this architecture uses 2 stages of 8-1 channel muxes instead.

SingleSW2ChUni

This architecture uses 6 stages of 2-1 channel muxes but combines SLL/SRL/SRA to all use the same barrelshifter using the unified bidirectional chain method to deal with SLL. 64-bit and 32-bit shifts are all done in the same barrel shifter using the swapword method.

SingleSW4ChUni

Similar to SingleSW2ChUni but this architecture uses 3 stages of 4-1 channel muxes instead.

Dual248ChUni

Similar to Dual2ChUni architecture but uses 2-1 channel mux as the first stage then a 4-1 channel mux for second stage then a 8-1 channel mux as the last stage. 32-bit shifts are handled separately using a 2-1 channel mux into two 4-1 channel muxes.

Dual824ChUni

Similar to Dual248ChUni architecture but the mux in each stage are ordered differently to 8, 2, and 4 rather than 2, 4, 8. For 32-bit the order of muxes are 2, 2, and 8.

SingleSW2ChSep

This architecture uses 6 stages of 2-1 channel muxes for 64-bit shifts and has 3 separate barrel shifters for SLL/SRL/SRA. 64-bit and 32-bit shifts are all done in the same barrel shifter using the swapword method.

SingleZU2ChSep

Similar to SingleSW2ChSep architecture but 64-bit and 32-bit shifts are all done in the same barrel shifter using the zero-upper method.

Topology	FPGA	Area (measured)	Area (LE equivalent)	Worst Propagation delay
Dual2ChUni	Cyclone IV	826 LEs	826 LEs	23 ns
Dual4ChUni	Cyclone IV	1086 LEs	1086 LEs	23.9 ns
Dual8ChUni	Cyclone IV	1070 LEs	1070 LEs	24.2 ns
SingleZU2ChRev	Cyclone IV	660 LEs	660 LEs	25.2 ns
SingleZU4ChRev	Cyclone IV	767 LEs	767 LEs	24.2 ns

SingleZU8ChRev	Cyclone IV	738 LEs	738 LEs	24.8 ns
SingleSW2ChUni	Cyclone IV	884 LEs	884 LEs	25.2 ns
SingleSW4ChUni	Cyclone IV	1081 LEs	1081 LEs	26.3 ns
Dual248ChUni	Cyclone IV	1081 LEs	1081 LEs	27.1 ns
Dual824ChUni	Cyclone IV	913 LEs	913 LEs	21.7 ns
SingleSW2ChSep	Cyclone IV	1149 LEs	1149 LEs	20.9 ns
SingleZU2ChSep	Cyclone IV	1161LEs	1161LEs	21.4 ns

Table 1. Worst-Case Propagation Delay for ShiftUnit Topology Variations on Cyclone IV

2.3 Top 3 ShiftUnit Topologies

After analyzing the resource utilization and worst-case propagation delays across the twelve variations, an initial attempt was made to construct a maximum-speed shifter by combining the two fastest architectures: **Dual824ChUni** and **SingleSW2ChSep**. However, timing analysis revealed that this hybrid approach produced significantly worse propagation delays than the standalone **SingleSW2ChSep** topology while consuming an additional 100 logic elements (LEs). Consequently, **SingleSW2ChSep** was retained as the high-speed candidate.

For the complexity-focused topology, the **SingleZU2ChRev** architecture was found to be highly optimized; requiring only 660 LEs, designing a more compact alternative proved impractical. **SingleZU2ChRev** was therefore selected as the complexity candidate. For the balanced topology, the **Dual2ChUni** architecture was chosen as an effective middle ground, utilizing 826 LEs with a propagation delay of 23 ns to bridge the gap between the speed- and complexity-focused implementations.

Speed - SwapUp2Ch

Complexity - Rev2Ch64

Balanced - DualChainBarrel

3 final topology will then be measured in arria II

Topology	FPGA	Area (measured)	Area (LE equivalent)	Worst Propagation delay
Dual2ChUni	Cyclone IV	826 LEs	826 LEs	23 ns
Dual2ChUni	Arria II	290 ALMs	690.2 LEs	17.7 ns
SingleZU2ChRev	Cyclone IV	660 LEs	660 LEs	25.2 ns
SingleZU2ChRev	Arria II	294 ALMs	699.72 LEs	18 ns
SingleSW2ChSep	Cyclone IV	1149 LEs	1149 LEs	20.9 ns
SingleSW2ChSep	Arria II	515 ALMs	1225.7 LEs	18.2 ns

Table 2. Comparison of the Three Selected ShiftUnit Topologies on Cyclone IV and Arria II

3. Subsystem Design Process LogicUnit

The LogicUnit is implemented as a purely combinational circuit parameterized by a generic N, defaulting to 64 bits. A single process sensitive to all inputs uses a case statement to decode the 2-bit LogicFN select signal, performing bitwise XOR, OR, or AND on operands A and B, or passing B directly to the output for LUI. The LUI operation requires no computational logic as the caller pre-zeroes the lower 12 bits of B before input. Another clause assigns 'X' for any undefined LogicFN input to aid simulation debugging.

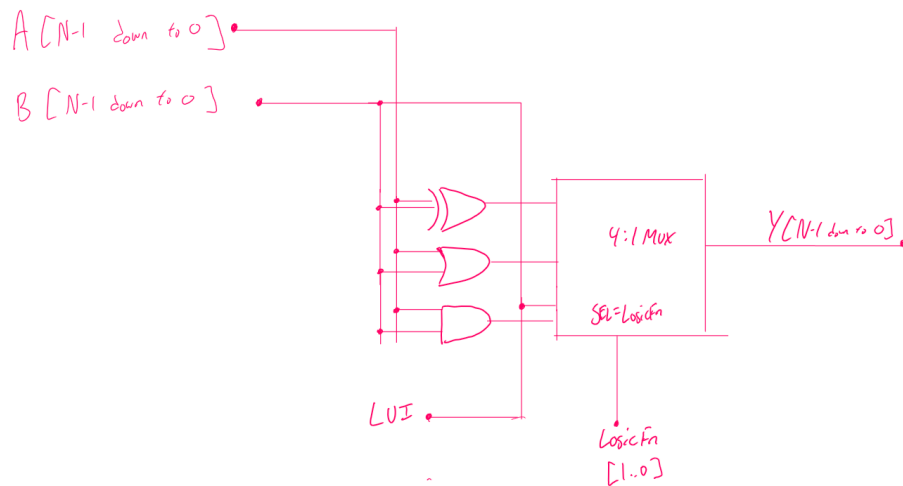


Figure 6. Hand-Drawn Diagram of LogicUnit

Port	Width	Direction	Description
A	64	in	Operand A
B	64	in	Operand B, also LUI passthrough
LogicFN	2	in	Function select
Y	64	out	Result

LogicFN	Operation	Output
00	LUI	B
01	XOR	A xor B
10	OR	A or B
11	AND	A and B

4. Design Process ExecUnit

The ExecUnit is the top-level entity that integrates the three subsystems(Arithmetic, Shift, and Logic) into a complete RV64I execution unit. All subsystem architectures are bound via VHDL configurations, meaning the ExecUnit source itself remains unchanged across all design candidates. A diagram of the top-level structure is shown in Figure X.

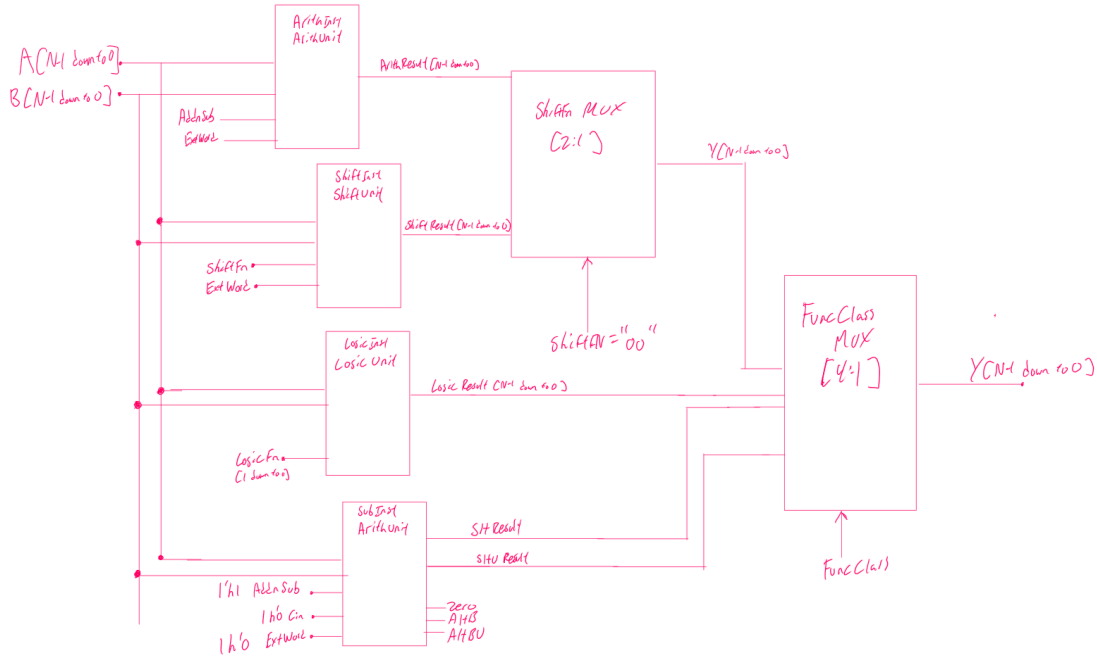


Figure 7. Hand-Drawn Diagram of ExecUnit Diagram

Subsystem Instantiations

The ExecUnit instantiates four components. ArithInst is the main arithmetic unit, computing $A+B$ or $A-B$ depending on AddSub, with its Cout and Ovfl outputs left open since these signals are not used at the ExecUnit level. ShiftInst computes SLL, SRL, or SRA based on ShiftFN. LogicInst computes AND, OR, XOR, or LUI based on LogicFN. SubInst is a second instantiation of the same ArithUnit entity with AddSub hardcoded to '1' and ExtWord hardcoded to '0', always computing $A-B$ regardless of FuncClass, used exclusively for status flag generation.

AddSub and ExtWord

Subtraction and 32-bit operation handling are both implemented inside the ArithUnit entity rather than in the ExecUnit itself. The ExecUnit passes AddSub and ExtWord directly through to ArithInst. Inside ArithUnit, AddSub='1' inverts B and forces Cin='1' to compute $A+(\sim B)+1 = A-B$ in two's complement. ExtWord='1' zeros the upper 32 bits of both operands before the operation and sign-extends the result from bit 31, matching RISC-V W-type instruction behaviour. Note that Cout and Ovfl are not valid for 32-bit operations. SubInst always receives AddSub='1' and ExtWord='0' directly, so it always performs a full 64-bit subtraction independent of the control inputs.

Status Flags

The three status flags are always derived from the SubInst result regardless of the active FuncClass. Zero is asserted when all 64 bits of Sub_Result are zero, computed via an OR-reduction of all bits followed by inversion. AltB is computed as $\text{Sub_Result}(N-1) \text{ XOR } \text{Sub_Ovfl}$, where the XOR corrects for signed overflow cases that would otherwise flip the sign bit incorrectly. AltBu is asserted when Sub_Cout='0', indicating a borrow occurred and A is unsigned-less-than B.

SLT and SLTU

SLT and SLTU do not require separate hardware. SLT places AltB_int into bit 0 of Y with all remaining bits zeroed, and SLTU does the same with AltBu_int, matching the RISC-V specification where these instructions write 1 or 0 to the destination register.

Output MUX

The output MUX uses two-level selection implemented as a case statement on FuncClass. FuncClass="01" routes LogicResult, FuncClass="10" routes SltResult, and FuncClass="11" routes SltuResult. When FuncClass="00", ShiftFN determines the final output: ShiftFN="00" routes ArithResult, while ShiftFN="01", "10", or "11" routes ShiftResult. Undefined inputs produce X on the output.

Port Description

Port	Width	Direction	Description
A	64	in	Operand A
B	64	in	Operand B
FuncClass	2	in	Selects output subsystem
LogicFN	2	in	Logic operation select
ShiftFN	2	in	Shift operation select
AddnSub	1	in	'0'=add, '1'=subtract
ExtWord	1	in	'1' for 32-bit mode
Y	64	out	Result output
Zero	1	out	A = B flag
AltB	1	out	A < B signed flag
AltBu	1	out	A < B unsigned flag

2. Testbenches and Test Vectors

2.1 Testbench Design

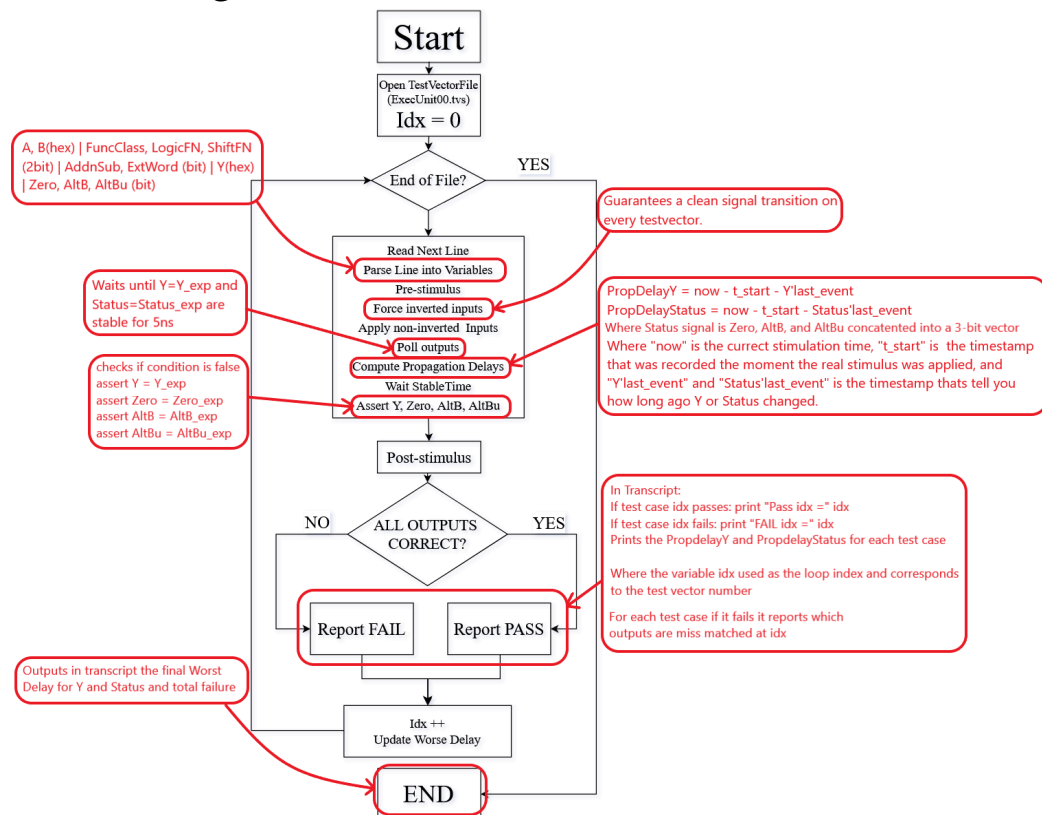


Figure 8. Flow Chart of Test Bench

A pre-stimulus duration of 40 ns and a post-stimulus duration of 100 ns were selected for the simulation. The 100 ns post-stimulus delay provides adequate settling time for signal propagation through all 64 bits across all adders, ensuring outputs stabilize before verification, while the 40 ns pre-stimulus 'X' state ensures clean transitions between consecutive test cases.

2.2 Test Vectors

Format of Test Vectors:

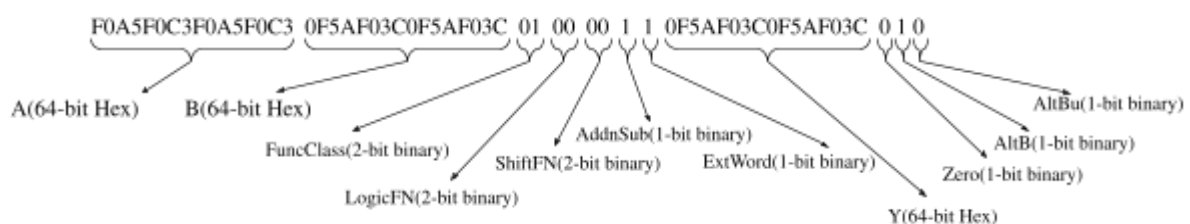


Figure 9. Test Vector Field Format for 64-bit ExecUnit Verification

Where A and B are 64-bit hexadecimal inputs, FuncClass, LogicFN, ShiftFN, AddnSub, and ExtWord are control signals, and Y, Zero, AltB, and AltBu are the expected outputs.

Test vectors were verified by manually calculating expected results for each test case to establish ground-truth reference values for the 64-bit output Y and all associated status flags. The test suite is organized into nine sections covering major RV64I execution unit operations, including arithmetic functions (ADD, SUB), shift operations (SLL, SRL, SRA), bitwise logical operations (LUI, XOR, OR, AND), set-less-than comparisons (SLT, SLTU), 32-bit word operations (ADDW, SUBW, SRAW), and worst-case timing vectors. Representative examples include:

Positive Signed Overflow (ADD):

7FFFFFFFFFFFFFFF 0000000000000001 00 00 00 0 0 8000000000000000 0 0 0

Subtraction with Borrow (SUB):

0000000000000000 0000000000000001 00 00 00 1 0 FFFFFFFFFFFFFFFF 0 1 1

Here are the worst-case timing vectors:

Full 64-bit Carry Ripple:

FFFFFFFFFFFFFFFF 0000000000000001 00 00 00 0 0 0000000000000000 0 1 0

Worst-Case Carry Select:

5555555555555555 5555555555555556 00 00 00 0 0 AAAAAAAAAAAAAAAB 0 1 1

SLL All 6 Stages (shift by 63):

0000000000000001 000000000000003F 00 00 01 0 0 8000000000000000 0 1 1

SRL All 6 Stages (shift by 63):

8000000000000000 000000000000003F 00 00 10 0 0 0000000000000001 0 1 0

SRA All 6 Stages All Ones (shift by 63):

FFFFFFFFFFFFFFFF 000000000000003F 00 00 11 0 0 FFFFFFFFFFFFFFFF 0 1 0

Worst-Case Carry Skip:

6666666666666666 1111111111111112 00 00 00 0 0 7777777777777778 0 0 1

Dedicated worst-case timing vectors were designed in Section 9 to stress the critical paths of the architecture. These vectors include a full 64-bit carry ripple, a worst-case carry-select pattern featuring alternating bit inputs, and shift-by-63 test cases across all three shift operations (SLL, SRL, and SRA) to ensure all six barrel-shifter stages are exercised simultaneously. The resulting test suite provides comprehensive coverage of functional correctness, edge cases, and timing performance across all ExecUnit functional blocks.

3. Results & Analysis

3.1 Waveform Analysis

All 8 design candidates were run through the same functional simulation testbench and passed with zero failures, confirming correct operation across all arithmetic, shift, logic, SLT/SLTU, and extended word operations for every topology. The CarrySelect + SingleSW2ChSep topology is used here as a representative example to illustrate the simulation output and transcript format that was observed consistently across all candidates.

The simulation applies all test vectors sequentially, with the test vector index (idx) visible in the waveform, incrementing across all 39 test cases. The waveform confirms that at each index transition, the output Y and all status flags (Zero, AltB, AltBu, Status) match their expected values, with no mismatches observed across any functional group.

The transcript confirms complete functional correctness for this candidate. At idx = 39, the final test vector is processed and the simulation reports PASS with PropDelayY = 0 ps and PropDelayFlag = 0 ps, indicating that by the time the testbench performs its final measurement, all outputs have already stabilized within the simulation timestep. The transcript then confirms that all test vectors have been processed with a total failure count of zero, verifying that the CarrySelect + SingleSW2ChSep design, along with all other candidates, produces correct results across the entire test suite.



Figure 10. CarrySelect + SingleSW2ChSep Functional Simulation

```

# Time: 5694 ns Iteration: 0 Instance: /tbexecunit
# ** Note: ** Note: PASS idx=39 PropDelayY=0 ps PropDelayFlag=0 ps
# Time: 5840 ns Iteration: 0 Instance: /tbexecunit
# ** Note: ** Note: All test vectors processed
# Time: 5840 ns Iteration: 0 Instance: /tbexecunit
# ** Note: ** Note: Total failures = 0
# Time: 5840 ns Iteration: 0 Instance: /tbexecunit

```

Figure 11. Transcript of CarrySelect + SingleSW2ChSep Functional Simulation

To evaluate the ExecUnit design space, timing simulations were conducted for all 8 design candidates across three operand widths: N = 64 implemented on both the Cyclone IV and Arria II FPGA targets. For each configuration, the worst-case propagation delay was recorded from the testbench transcript and waveform.

To illustrate the measurement methodology, consider the CarrySelect + SingleSW2ChSep timing simulation at N = 64 on the Cyclone IV (Figure X). The testbench applies input vectors sequentially and records both PropDelayY, the time for the output Y to stabilize, and PropDelayFlag, the time for all status flags to reach their final values. At test index idx = 2, the worst-case delays are observed: PropDelayY = 31,908 ps and PropDelayFlag = 10,601 ps. The transcript confirms PASS results across all test vectors, with the worst-case PropDelayY corresponding to the reported worst-case propagation delay of 31,908 ps. This same measurement procedure was applied uniformly across all 8 candidates and both FPGA platforms at each operand width, with the resulting worst-case propagation delays compiled and compared in Section X.

ADD: positive overflow

```
7FFFFFFFFFFFFFFFFF 0000000000000001 00 00 00 0 0 8000000000000000 0 0 0
```

ADD: all ones

```
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF 00 00 00 0 0 FFFFFFFFFFFFFFFF 1 0 0
```

The worst-case propagation delay at idx = 2 corresponds to the test vector A = 0x7FFFFFFFFFFFFFFFFF, B = 0x0000000000000001, with an ADD operation. This represents a signed positive overflow case: A is the largest representable positive 64-bit signed integer ($2^{63} - 1$), and adding 1 forces a carry to propagate through all 63 lower bit positions before resolving at the most significant bit, producing a result of 0x8000000000000000. This maximum-length carry chain is the condition that stresses the critical path of any carry-propagating adder, as no intermediate carry can be absorbed or killed. Therefore, every stage must wait for the carry from the stage below it to settle before it can produce its own output. It is therefore expected that this vector produces the longest PropDelayY observed across all test cases.

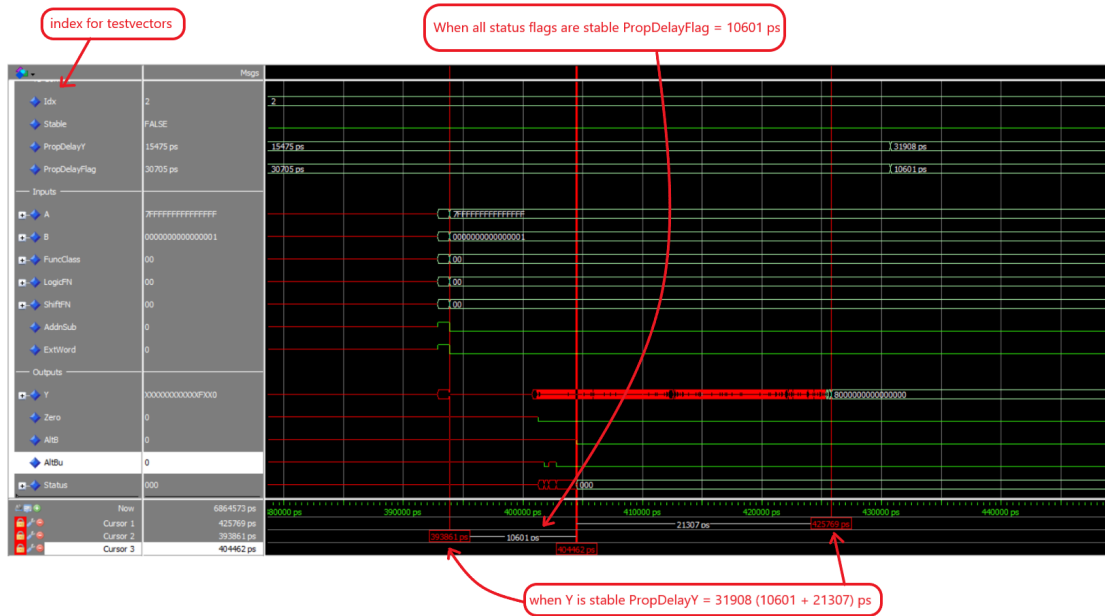


Figure 12. Worst PropDelayY CarrySelect + SwapUp Timing Simulation

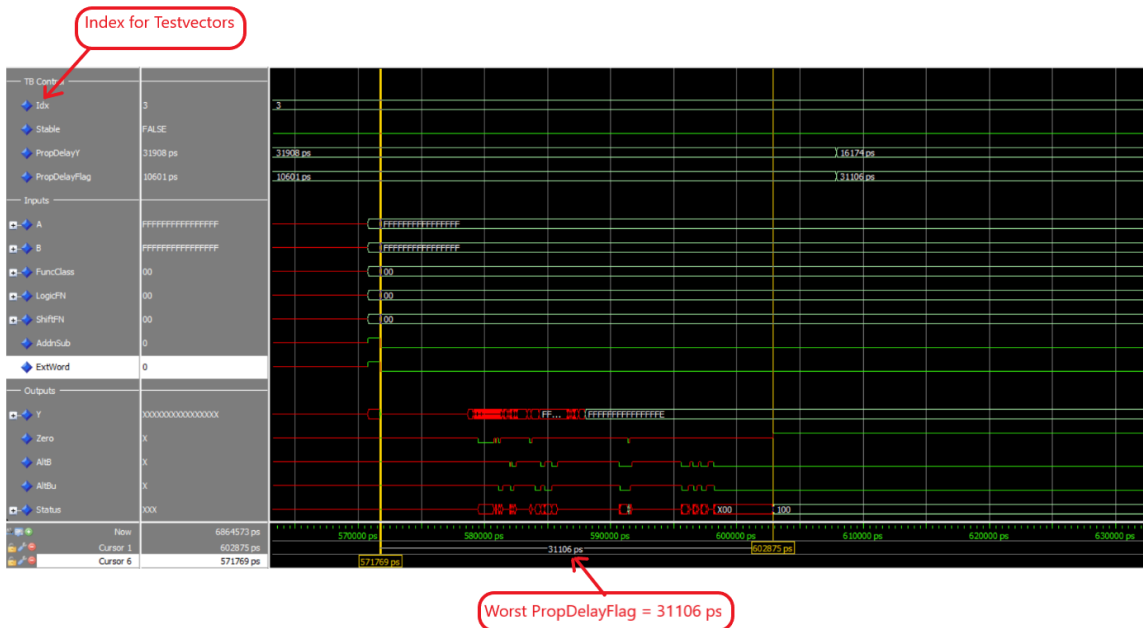


Figure 13. Worst PropDelayFlag CarrySelect + SwapUp Timing Simulation



Figure 14. Transcript of CarrySelect + SwapUp Timing Simulation

3.2 Predictions

3.2.1 LEs and ALMs

3.2.1.1 ArithUnit

Baseline Ripple Adder - Cyclone IV

In the Cyclone IV, each Logic Element (LE) contains a 4-input LUT. Due to this input limitation, a structurally implemented full adder requires approximately 2 LUTs: one for the sum and one for the carry logic. This sums to 2 LEs per bit, resulting in:

$$64 \times 2 = 128 \text{ LEs}$$

for a 64-bit structural ripple adder.

Conditional Sum Adder - Cyclone IV

For the conditional sum architecture, three adders are instantiated recursively (a lower block and two upper blocks). Thus, at the 64-bit level, three 32-bit adders are required, with each 32-bit adder containing three 16-bit adders, continuing recursively until reaching the base case of a 1-bit adder.

Additionally, 33 MUXes (32 for the upper sum and 1 for Cout) are required at the 64-bit level, 17 at the 32-bit level, and so on. Assuming that a 1-bit 2-to-1 MUX requires 1 LUT and a 1-bit adder requires 2 LEs (one for Cout and one for Sum), the resulting resource utilization is calculated as follows:

$$\begin{aligned}
C_2 &= 3 \cdot C_1 + 2 \cdot MUX = 8 \text{ LEs} \\
C_4 &= 3 \cdot C_2 + 3 \cdot MUX = 27 \text{ LEs} \\
C_8 &= 3 \cdot C_4 + 5 \cdot MUX = 86 \text{ LEs} \\
C_{16} &= 3 \cdot C_8 + 9 \cdot MUX = 267 \text{ LEs} \\
C_{32} &= 3 \cdot C_{16} + 17 \cdot MUX = 818 \text{ LEs} \\
C_{64} &= 3 \cdot C_{32} + 33 \cdot MUX = 2487 \text{ LEs}
\end{aligned}$$

for a 64-bit conditional sum adder.

Block Carry Look-Ahead Adder - Cyclone IV

For block carry look-ahead adder, the design uses the carry look-ahead formula where each block contains

$$\begin{aligned}
C_1 &= G_0 + P_0 C_0 \\
C_2 &= G_1 + G_0 P_1 + P_1 P_0 C_0 \\
C_3 &= G_2 + G_1 P_2 + G_0 P_2 P_1 + P_2 P_1 P_0 C_0
\end{aligned}$$

The carry signal C_1 requires 1 LUT, and the number of LUTs required for C_2 using the formula $C_2 = G_1 + P_1 C_1$. Because this expression takes the same form as C_1 with C_1 serving as the input instead of C_0 , C_2 requires 2 LUTs. Extending this principle to C_3 results in a requirement of 3 LUTs. Additionally, each block produces four sum outputs, requiring 1 LUT per sum for a total of 4 LUTs. Each block also contains:

$$\begin{aligned}
P_{block,k} &= P_3 P_2 P_1 P_0 \\
G_{block,k} &= G_3 + G_2 P_3 + G_1 P_3 P_2 + G_0 P_3 P_2 P_1 \\
C_{k+1} &= G_{Block,k} + P_{Block,k} C_k
\end{aligned}$$

Additionally, 1 LUT is required for the P_{block} and 3 LUTs for the G_{block} as it follows the same concept as C_3 , for a total of 5 LUTs for C_k . Thus, the total number of LUTs required for a 16-block carry look-ahead adder is

$$\begin{aligned}
&16 \cdot (C_k + 4Sum + C_1 + C_2 + C_3) \\
&16 \cdot (5 + 4 + 1 + 2 + 3) = 240 \text{ LEs}
\end{aligned}$$

Carry Skip Adder - Cyclone IV

For the carry-skip adder, 16 blocks are utilized, where each block consists of a 4-bit ripple adder that also calculates a block propagate signal and the carry-skip formula.

$$P_{block,k} = P_3 P_2 P_1 P_0$$

$$C_{out} = P_{block} \cdot C_{in} + \overline{P_{block}} \cdot C_{ripple}$$

Where the propagation block requires 1 LEs and the 4-bit ripple adder requires 8 LEs and the carry skip formula requires 1 LEs as it is just a 1-bit 2-1 MUX. Thus the total number of LEs for the carry skip adder is

$$16 * (8 + 1 + 1) = 160 \text{ LEs}$$

Carry Select Adder - Cyclone IV

The carry-select adder is partitioned into seven unequal blocks of 4, 4, 8, 8, 16, 16, and 8 bits. Except for the initial block, all blocks are implemented using two parallel adders. Because each baseline full-adder bit requires 2 logic elements (LEs), the initial 4-bit block consumes 8 LEs. Each subsequent N-bit block requires two N-bit adders alongside N + 1 2-to-1 multiplexers for selecting the N sum bits and the final carry-out. Assuming each 2-to-1 multiplexer requires 1 LE, an N-bit block consumes 5N + 1 LEs. Thus, the total resource utilization for the carry-select adder is 314 LEs.

$$\begin{aligned} MUXes &= 5 + 9 + 9 + 17 + 17 + 9 = 66 \\ LEs &= 2 * (C_4 + 2 * (C_4 + C_8 + C_8 + C_{16} + C_{16} + C_8)) + 66 \\ &= 2 * (4 + 2 * (4 + 8 + 8 + 16 + 16 + 8)) + 66 = 314 \text{ LEs} \end{aligned}$$

In contrast, the Arria II architecture utilizes Adaptive Logic Modules (ALMs), where each ALM features an 8-input Adaptive Look-Up Table (ALUT) capability that effectively doubles the functional capacity of a standard 4-input LUT in a logic element (LE). Consequently, 1 ALUT is estimated to be equivalent to 2 LUTs or 2 LEs. The predicted ALM utilization for the Arria II device is summarized below:

Conditional Sum Adder - Arria II

$$\frac{2487}{2} = 1243.5 \text{ ALMs}$$

Block Carry Look-Ahead Adder - Arria II

$$\frac{240}{2} = 120 \text{ ALMs}$$

Carry Skip Adder - Arria II

$$\frac{160}{2} = 80 \text{ ALMs}$$

Carry Select Adder - Arria II

$$\frac{314}{2} = 157 \text{ ALMs}$$

3.2.1.2 ShiftUnit

Dual2ChUni - Cyclone IV

To predict the LE count for a barrel shifter supporting SRL, SLL, and SRA across both 32-bit and 64-bit instruction widths in a single unified design with 6 stages of 2:1 muxes, the calculation begins with the core datapath. A 64-bit shifter with 6 stages requires $6 \times 64 = 384$ 2:1 muxes for the main data path. Since the same hardware must also handle 32-bit instructions, a practical implementation requires a parallel 32-bit chain of 5 stages, contributing a further $5 \times 32 = 160$ muxes, for a combined raw mux count of 544. Each 2:1 mux in isolation maps cleanly to 1 LE, giving a baseline of 544 LEs. However, because the shifter must handle three operations SRL, SLL, and SRA each stage mux must additionally be conditioned on a direction control signal for SLL versus SRL/SRA and a fill signal for SRA arithmetic sign extension, introducing at minimum 2 extra control inputs per mux function. This pushes each bit's logic function from 2 inputs to 4–5 inputs, and since a 4-input LUT is the LE unit, functions exceeding 4 inputs spill into a second LUT, raising the average cost per mux to approximately 1.5 LEs, giving $544 \times 1.5 = 816$ LEs for the core datapath.

SingleZU2ChRev - Cyclone IV

For a combined single-chain design used to merge 32-bit and 64-bit instruction handling into a single barrel shifter, the prediction changes significantly. The core datapath now consists of only one chain operating at the full 64-bit width across 6 stages, giving $6 \times 64 = 384$ raw 2:1 muxes as the baseline. The same 1.5 LE average cost per mux applies for the same reasoning as before, the three operation modes SRL, SLL, and SRA introduce direction and fill control signals that push boundary bits beyond 4 inputs while non-boundary bits remain at 4 inputs, averaging out to 1.5 LEs per mux across the distribution, giving $384 \times 1.5 = 576$ LEs for the entire datapath. The 32-bit mode is handled within this same chain by clamping the shift amount to 5 bits for 32-bit instructions rather than running a parallel chain, so no additional mux array is needed. Thus the total LE count is 576.

SingleSW2ChSep - Cyclone IV

For a design where SLL, SRL, and SRA is split into three dedicated independent shifters, each combined with to handle both 32-bit and 64-bit instructions within a single chain per operation, the prediction is calculated as follows. Each dedicated shifter operates on a single 64-bit wide chain across 6 stages, giving $6 \times 64 = 384$ muxes per shifter, and with three shifters the total raw mux count is $3 \times 384 = 1152$ muxes. The 32-bit mode is handled within each chain by shift amount clamping, adding no additional mux arrays. Thus the total LE count is 1152.

Dual2ChUni - Arria II

$$\frac{816}{2} = 408 \text{ ALMs}$$

SingleZU2ChRev - Arria II

$$\frac{576}{2} = 288 \text{ ALMs}$$

SingleSW2ChSep - Arria II

$$\frac{1152}{2} = 576 \text{ ALMs}$$

3.2.1.3 LogicUnit

LogicUnit - Cyclone IV

For a design where a single logic unit handles AND, XOR, OR, and LUI operations across a 64-bit datapath. LUI requires no computational LEs as it is resolved entirely by output selection. Each of the three computational operations is bitwise and fully parallel, with each bit being a simple 2-input function of A and B fitting into a single LUT at 1 LE per bit, giving $3 \times 64 = 192$ LEs.

LogicUnit - Arria II

$$\frac{192}{2} = 96 \text{ ALMs}$$

3.2.1.3 ExecUnit

Since the ExecUnit consists of ArithUnit, ShiftUnit, and LogicUnit, the formula for predicted LEs for the ExecUnit is ArithUnit + ShiftUnit + LogicUnit.

CarrySelect + SingleSW2ChSep - Cyclone IV

$$314 + 1152 + 192 = 1658 \text{ LEs}$$

CarrySelect + SingleSW2ChSep - Arria II

$$147 + 576 + 96 = 829 \text{ ALMs}$$

CarrySkip + SingleZU2ChRev - Cyclone IV

$$160 + 576 + 192 = 928 \text{ LEs}$$

CarrySkip + SingleZU2ChRev - Arria II

$$80 + 288 + 96 = 464 \text{ ALMs}$$

BCLA + Dual2ChUni - Cyclone IV

$$240 + 816 + 192 = 1248 \text{ LEs}$$

BCLA + Dual2ChUni - Arria II

$$120 + 408 + 96 = 624 \text{ ALMs}$$

ConditionalSum + SingleSW2ChSep - Cyclone IV

$$2487 + 1152 + 192 = 3831 \text{ LEs}$$

ConditionalSum + SingleSW2ChSep - Arria II

$$1243.5 + 576 + 96 = 1915.5 \text{ ALMs}$$

3.2.2 Timing Delays

3.2.2.1 ArithUnit

Baseline Ripple Adder

According to the Cyclone IV Device Datasheet (Table 1–24), the maximum core clock frequency of the EP4CE115F29C7 device is 437.5 MHz, giving a minimum period of 2.286 ns. This is used as the per-LUT-level delay estimate for the baseline ripple adder calculation.

Since there are 64 levels of LUT in the 64-bit ripple adder the worst sum and cout delay are

$$\text{Worst Sum Delay} = 2.286 \text{ ns} * 64 = 146.3 \text{ ns}$$

$$\text{Worst Cout Delay} = 2.286 \text{ ns} * 64 = 146.3 \text{ ns}$$

$$\text{Worst Status Delay} = \text{Worst Cout Delay} + 2.286 = 148.59 \text{ ns}$$

Since overflow is calculated by $C_{64} \text{ XOR } C_{63}$ the worst status delay is 1 LUT more than the worst cout delay.

Conditional Sum Adder - Cyclone IV

Assuming both a 1-bit 2-1 MUX and 1-bit adder only requires 1 LUT the delay of a MUX and 1-bit adder is 2.286 ns.

$$D_{64} = D_{32} + D_{MUX}$$

$$D_{32} = D_{16} + D_{MUX}$$

$$D_{16} = D_8 + D_{MUX}$$

$$D_8 = D_4 + D_{MUX}$$

$$D_4 = D_2 + D_{MUX}$$

$$D_2 = D_1 + D_{MUX}$$

$$Worst\ Sum\ Delay = D_{64} = D_1 + 6 D_{MUX} = 16\ ns$$

$$Worst\ Cout\ Delay = D_{64} = D_1 + 6 D_{MUX} = 16\ ns$$

Since ovfl delay requires another LUT after the last sum the worse status delay is:

$$Ovfl = \overline{(A_{N-1} \oplus B_{N-1})} \cdot (S_{N-1} \oplus A_{N-1})$$

$$Worst\ Ovfl\ Delay = Worst\ Sum\ Delay + 2.286\ ns = 18.286\ ns$$

$$Worst\ Status\ Delay = 18.286\ ns$$

Block Carry Look-Ahead Adder

For the block carry look-ahead adder each block is a 4-bit carry look-ahead adder so the propagate and generate have a 1 LUT level of delay.

$$P_i = A_i \oplus B_i, G_i = A_i \cdot B_i$$

C₁ and C₂ have a 1 LUT level delay while C₃ has a 2 LUT level delay as it requires 2 LUTs to complete its logic as it has more than 4 inputs.

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + G_0 P_1 + P_1 P_0 C_0$$

$$C_3 = G_2 + G_1 P_2 + G_0 P_2 P_1 + P_2 P_1 P_0 C_0$$

All P blocks have a 1 LUT level delay and all G blocks have a 2 LUT level delay. The cout from the blocks will have a 1 LUT level delay.

$$P_{block,k} = P_3 P_2 P_1 P_0$$

$$G_{block,k} = G_3 + G_2 P_3 + G_1 P_3 P_2 + G_0 P_3 P_2 P_1$$

$$C_{k+1} = G_{Block,k} + P_{Block,k} C_k$$

The carry out from the 15th block is required to calculate the worst Sum delay from $S_i = P_i \oplus C_i$ using internal carry C_3 from the last block. Thus, the first block has 3 level LUT delay, then 1 LUT delay for every carry chain after. Then it has to go through the internal C_3 from the last block which has a 2 LUT delay then into the final Sum calculation LUT delay.

$$Worst\ Sum\ Delay = 2.286 * (1 + 2 + 14 + 3) = 45.73\ ns$$

Since overflow is calculated using C_{16} xor C_{15} the worst status delay is 1 LUT delay more than the worst carry out delay

$$Worst\ Cout\ Delay = 2.286 * (3 + 15) = 41.1\ ns$$

$$Worst\ Status\ Delay = Worst\ Cout\ Delay + 2.286 = 43.434\ ns$$

Carry Skip Adder

For the block carry skip adder each block is a 4-bit carry look-ahead adder so the propagate and generate have a 1 LUT level of delay.

$$P_i = A_i \oplus B_i, G_i = A_i \cdot B_i$$

Within each 4-bit block, the carry ripples so to calculate the internal cout, C_4 , of a block it must traverse the ripple chain.

$$C_{i+1} = G_i + P_i C_i$$

C_1 and C_2 have a 1 LUT level delay while C_3 and C_4 have a 2 LUT level delay as it requires 2 LUTs to complete its logic as it has more than 4 inputs. All P blocks have a 1 LUT level delay.

$$P_{block,k} = P_3 P_2 P_1 P_0$$

$$C_{out} = P_{block} \cdot C_{in} + \overline{P_{block}} \cdot C_4$$

The first block requires 3 LUT levels to produce the first valid Cout (1 for P_i/G_i , 2 for internal c_4 ripple, and 1 for the skip MUX). Each subsequent block adds 1 LUT level of delay as the carry passes through the skip MUX. The carry arrives at the 16th block, then must ripple internally to c_3 (2 LUT levels) and finally into the Sum calculation LUT (1 level).

$$Worst\ Sum\ Delay = 2.286 * (1 + 2 + 14 + 3) = 45.73\ ns$$

Since overflow is calculated using C_{64} xor C_{63} the worst status delay is 1 LUT delay more than the worst carry out delay

$$\begin{aligned} \text{Worst Cout Delay} &= 2.286 * (3 + 15) = 41.1 \text{ ns} \\ \text{Worst Status Delay} &= \text{Worst Cout Delay} + 2.286 = 43.434 \text{ ns} \end{aligned}$$

Carry Select Adder

For the carry select adder with 7 unequal blocks of 4, 4, 8, 8, 16, 16, and 8 bits and assuming a 1-bit 2-1 MUX has a 1 LUT level delay. The bottleneck of this carry select adder would be the first 16-bit ripple adder, as the previous blocks must wait for this block to finish to continue. Thus, the worst sum delay of the carry select adder would be a 16-bit ripple adder, then 3 MUXes(1 from first 16-bit + 1 from second 16-bit + 1 last 8-bit) as the carry out from the first 16-bit ripple adder must go through the rest of the MUXes to get the final value.

$$\text{Worst Sum Delay} = (16 \cdot 2.286) + (3 \cdot 2.286) = 43.4 \text{ ns}$$

Since overflow is calculated using $C_{64} \text{ xor } C_{63}$, the worst status delay is 1 LUT level delay more than the worst carry out delay. The worst cout delay is calculated at the same time as the sum delay, as they both depend on the same selector delay(Cout).

$$\begin{aligned} \text{Worst Cout Delay} &= \text{Worst Sum Delay} = 43.4 \text{ ns} \\ \text{Worst Status Delay} &= \text{Worst Cout Delay} + 2.286 = 45.7 \text{ ns} \end{aligned}$$

3.2.2.1 ShiftUnit

Dual2ChUni

To predict the worst case timing delay for this barrel shifter, the critical path runs through the longest chain of sequential logic from input to output. The worst case path is through the 64-bit chain since it has 6 stages compared to the 32-bit chain which only has 5 stages. Each stage consists of a single 2-1 channel mux, but because the design combines SRL, SLL, and SRA into one shifter, each mux function has the direction control is_left and fill signal as additional inputs alongside B(i) and the data, pushing some bits to a 2-LUT implementation. The worst case delay per stage is therefore 2 LUT delays for the boundary bits that spill into a second LUT, rather than 1. The critical path delay is then calculated as 6 stages $\times$ 2 LUT delays = 12 LUT delays through the mux chain, plus the input logic delay for computing the fill signal from the sign bit for SRA which adds 1 LUT delay at the front, and the final output mux selecting between the 32-bit and 64-bit results which adds 1 LUT delay at the end, giving a total critical path depth of 14 LUT delays.

$$\text{Worst Y Delay} = 14 * 2.286 = 32 \text{ ns}$$

SingleZU2ChRev

For this prediction the critical path is calculated similarly to the previous architecture's, Dual2ChUni, design. Running through the 64-bit chain across 6 stages with each stage costing 2 LUT delays for boundary bits due to the combined SRL, SLL, and SRA control signals, plus 1 LUT delay at the front for the fill signal computation and 1 LUT delay for the final output mux, giving 14 LUT delays. However, because the 32-bit and 64-bit instructions share the same chain, the 32-bit result must pass through an additional LUT level to select between the 32-bit and 64-bit interpretation of the chain output, adding 1 further LUT delay and bringing the total critical path depth to 15 LUT delays

$$Worst\ Y\ Delay = 15 * 2.286 = 34.3\ ns$$

SingleSW2ChSep

To predict the worst case timing delay for this design with separate SRL, SRA, and SLL barrel shifters each with dedicated 32 and 64-bit chains, the critical path is simpler than the combined operation design because each shifter only handles one operation, meaning the direction control and fill signals are hardwired constants rather than variable inputs, reducing every stage mux back to a 1-LUT implementation. The critical path runs through the 64-bit chain of the SRA shifter which is the worst case of the three since it requires an additional 1 LUT delay at the front for gating the fill signal with the sign bit, giving $1 + 6 = 7$ LUT delays in total. Since each shifter has a dedicated chain for each instruction width there is no final output mux on the critical path.

$$Worst\ Y\ Delay = 7 * 2.286 = 16\ ns$$

3.2.2.1 ExecUnit

The worst case delay of the ExecUnit is dependent on the instruction being executed, as different instruction types traverse different datapaths with varying critical path depths. Since the ExecUnit combines a ShiftUnit and an ArithUnit, the worst case delay of the entire ExecUnit is determined by whichever of the two units produces the greater critical path delay. The LogicUnit delay is omitted from this analysis as its single-level LUT operations produce a significantly shorter critical path than both shift and arithmetic instructions, meaning it will never be the limiting factor in determining the ExecUnit worst case delay. Therefore, predicting the worst case delay of the ExecUnit reduces to independently predicting the worst case delay of each ShiftUnit and ArithUnit topology and taking the greater of the two as the ExecUnit worst case delay.

CarrySelect + SingleSW2ChSep

$$Worst\ Delay = 43.4\ ns$$

CarrySkip + SingleZU2ChRev

$$WorstDelay = 41.1\ ns$$

BCLA + Dual2ChUni

$$WorstDelay = 41.1 \text{ ns}$$

ConditionalSum + SingleSW2ChSep

$$Worst Delay = 16 \text{ ns}$$

3.3 Cost Analysis

Cost per LE:

The following devices were used:

- EP4CE115F29C7 (Cyclone IV)
- EP2AGX45DF29C6 (ARRIA II)

EP4CE115F29C7 (Cyclone IV):

From DigiKey, the EP4CE115F29C7 device contains 114,480 LEs and on February 28th, 2026, is priced at \$1020.20 CAD per unit. This gives an estimated cost of \$0.008912 per LE.

EP2AGX45DF29C6 (ARRIA II):

Sourcing from the Digikey datasheet, one unit consists of 18,050 ALMs and one unit costs \$1261.59 CAD on February 28th, 2026. This means that the cost per ALM is \$0.069894.

Conversion:

The key architectural difference between Cyclone IV and Arria II devices is that Cyclone IV uses Logic Elements (LEs), while Arria II uses Adaptive Logic Modules (ALMs) and ALUTs. Since this project defines the Cyclone IV LE as the standard unit of cost, Arria II resource usage must be converted into equivalent LE units for fair comparison.

Using the cost per ALM/LE from the preceding section, a conversion factor of:

$$Conversion Factor = \frac{\$ \text{ per ALM}}{\$ \text{ per LE}} = \frac{0.069894}{0.008912} = 7.836 \text{ LEs per ALM}$$

The calculated conversion factor of 7.836 LEs per ALM is unrealistic because an Arria II ALM is equivalent to approximately 2 LEs on the upper end (1 ALM = 1 ALUTs, 1 ALUT = 2 LEs). The inflated result likely arises from comparing devices with significantly different feature sets and performance capabilities, rather than isolating logic density alone. Therefore, the raw cost-per-logic calculation overestimates the effective conversion factor.

A more realistic conversion factor would be to use the given information in the data sheet for Arria II. The EP2AGX45DF29C6 device consists of 18,050 ALMs, which is equivalent to 42,959 LEs specified in the datasheet.

$$Conversion Factor = \frac{42959 \text{ LEs}}{18050 \text{ ALMs}} = 2.38 \text{ LEs per ALM}$$

Dividing the equivalent number of LEs by the number of ALMs provides a more realistic conversion factor because it isolates logic capacity rather than market pricing.

The following table of values comes from Quartus -> Compilation Report -> Flow Summary and Quartus -> Compilation Report -> Fitter -> Resource Section

$$\text{Normalized Cost} = \frac{\text{Device Area (LE)}}{\text{Baseline Area (LE)}}$$

Where Baseline is 161 LEs

Candidate (Arith Unit + Shift Unit)	FPGA	Area (measured)	Area (Predicted)	Area (LE equivalent)	Normalized Cost
CarrySelect + SingleZU2ChSep	Cyclone IV	1881 LEs	1658 LEs	1881 LEs	11.76
CarrySelect + SingleZU2ChSep	Arria II	825 ALMs	829 ALMs	1963.5 LEs	12.27
CarrySkip + SingleZU2ChRev	Cyclone IV	1227 LEs	928 LEs	1227 LEs	7.67
CarrySkip + SingleZU2ChRev	Arria II	530 ALMs	464 ALMs	1261.4 LEs	7.88
BCLA + Dual2ChUni	Cyclone IV	1523 LEs	1248 LEs	1523 LEs	9.52
BCLA+ Dual2ChUni	Arria II	646 ALMs	624 ALMs	1537.48 LEs	9.61
ConditionalSum + SingleZU2ChSep	Cyclone IV	2044 LEs	3831 LEs	2044 LEs	12.78
ConditionalSum + SingleZU2ChSep	Arria II	915 ALMs	1915.5 ALMs	2177.7 LEs	13.61

Table 3. Normalized Cost Comparison of ExecUnit Design Candidates

3.4 Timing Analysis

Candidate (Arith Unit + Shift Unit)	FPGA	Worst Propagation Delay (Predicted)	Worst Propagation Delay
--	-------------	--	--

CarrySelect + SingleZU2ChSep	Cyclone IV	43.4 ns	31.909 ns
CarrySelect + SingleZU2ChSep	Arria II	43.4 ns	26.115 ns
CarrySkip + SingleZU2ChRev	Cyclone IV	41.1 ns	82.682 ns
CarrySkip + SingleZU2ChRev	Arria II	41.1 ns	35.780 ns
BCLA + Dual2ChUni	Cyclone IV	41.1 ns	50.296 ns
BCLA+ Dual2ChUni	Arria II	41.1 ns	26.938 ns
ConditionalSum + SingleZU2ChSep	Cyclone IV	16 ns	40.939 ns
ConditionalSum + SingleZU2ChSep	Arria II	16 ns	31.953 ns

Table 4. Predicted and Measured Worst-Case Propagation Delay for ExecUnit Design Candidates

3.5 Interpreting Results

3.5.1 Resource Usage Predicted vs Measured

The predicted resource usage for the ExecUnit is derived by summing the individual lower-bound architectural estimates for the ArithUnit, ShiftUnit, and LogicUnit sub-units. The LogicUnit contributes a fixed 192 LEs (96 ALMs on Arria II) regardless of the adder or shifter topology, as it consists of three fully parallel 64-bit bitwise operations each mapping directly to one LUT per bit. The overall trends observed in the ExecUnit predicted vs measured comparison mirror those discussed for the ArithUnit in isolation, since the ShiftUnit and LogicUnit predictions are well-behaved and consistent across all candidates any major discrepancies between predicted and measured ExecUnit area are therefore driven primarily by the behaviour of the adder topology chosen.

The predicted resource usage for the ShiftUnit represents a lower-bound estimate, as the predictions account only for the raw mux logic within the barrel shifter datapath and do not capture routing overhead, fan-out penalties, or non-ideal LUT packing that Quartus introduced during synthesis. For the Cyclone IV results, the predictions consistently underestimate the measured area, confirming the lower-bound nature of the estimates:

Dual2ChUni measured 826 LEs against a prediction of 816 LEs (difference of 10), SingleZU2ChRev measured 660 LEs against 576 LEs (difference of 84), and SingleSW2ChSep measured 1149 LEs against 1152 LEs, which is effectively exact. The Arria II results behave differently, with measured ALM counts falling below the predicted values in most cases: Dual2ChUni measured 290 ALMs against 408 ALMs predicted, and SingleSW2ChSep measured 515 ALMs against 576 ALMs predicted. This is expected and does not contradict the lower-bound principle the Arria II predictions were derived by halving the Cyclone IV LE predictions using a simple $1 \text{ ALM} = 2 \text{ LEs}$ conversion, which is a conservative approximation. In practice, the Arria II ALM's 8-input fracturable LUT structure allows Quartus to pack logic significantly more efficiently than a straightforward halving would suggest, meaning the 1:2 ratio overstates the true ALM cost and the measured values fall below the converted predictions as a result.

The ConditionalSum + SingleZU2ChSep candidate exhibits the most dramatic discrepancy, and in the opposite direction to all other candidates. The predicted area was 3831 LEs on Cyclone IV and 1915.5 ALMs on Arria II, yet the measured ExecUnit areas were only 2044 LEs and 915 ALMs respectively. As established in the ArithUnit analysis, this inversion is caused entirely by the conditional-sum adder's recursive structure: the theoretical gate count grows exponentially with recursion depth, but Quartus identifies and eliminates the resulting redundant logic through aggressive logic minimization, collapsing the adder to a far smaller measured footprint than the architectural estimate predicts. The ShiftUnit and LogicUnit components behave normally and contribute their expected area; the anomaly is localised to the adder sub-unit, confirming that the ExecUnit-level discrepancy for this candidate is inherited directly from the conditional-sum adder's synthesis behaviour rather than from any ExecUnit-level integration effect.

3.5.2 Timing Delay Predicted vs Measured

The predicted worst-case propagation delay for the ExecUnit is determined by taking the greater of the ArithUnit and ShiftUnit predicted delays. The LogicUnit is excluded from this comparison as its single-level LUT operations bitwise AND, OR, and XOR across a 64-bit datapath produce a critical path of just one LUT delay (2.286 ns on Cyclone IV), which is negligibly small compared to the multi-stage arithmetic and shift datapaths and will never be the ExecUnit's worst-case path regardless of the candidate topology chosen. For all candidates except ConditionalSum, the ArithUnit predicted delay equals or exceeds the ShiftUnit predicted delay, meaning the adder sets the predicted critical path in every case. Except for ConditionalSum, the predicted delay is only 16 ns, far below the ShiftUnit predictions of 32 ns depending on topology.

For the CarrySelect + SingleZU2ChSep candidate on Cyclone IV, the predicted worst-case delay was 43.4 ns, driven by the CarrySelect adder, while the measured ExecUnit delay was 31.909 ns a significant overestimate of 11.5 ns. This is consistent with the same logic minimization behaviour observed in the area analysis: Quartus identifies and collapses redundant carry-select logic, shortening the realized critical path below what the gate-level model predicts. On Arria II the measured delay was

26.115 ns, which is faster still, reflecting the more efficient routing and higher-speed of the Arria II device.

For the CarrySkip + SingleZU2ChRev candidate on Cyclone IV, the predicted worst-case delay was 41.1 ns, while the measured delay was 82.682 ns, more than double the prediction. This is the largest discrepancy across all candidates and is notable because CarrySkip is generally considered a fast architecture. The measured delay being so much larger than predicted suggests that the synthesis of the combined ArithUnit and ShiftUnit introduced a significantly longer critical path than either sub unit prediction accounted for in isolation, likely due to routing congestion or carry chain interactions that are invisible to a gate-level estimate. On Arria II the measured delay was 35.780 ns, which is far more in line with expectations, suggesting the Cyclone IV result may reflect a placement and routing artefact specific to that device.

For the ConditionalSum + SingleZU2ChSep candidate on Cyclone IV, the predicted ArithUnit delay was only 16 ns due to the shallow recursive critical path of the conditional-sum adder, making the ShiftUnit the predicted bottleneck. However, the measured ExecUnit delay was 40.939 ns. As confirmed by timing analysis, the worst-case instruction measured across all candidates was always an arithmetic operation such as Add or Sub rather than a shift instruction. This indicates that despite the conditional-sum adder's theoretically shallow path, Quartus's synthesis of the full ExecUnit results in the adder datapath remaining the timing bottleneck in practice, likely because the recursive structure that allows logic minimization to reduce area also introduces additional mux-level delays through the selection tree that are not captured by the simple recursive depth model used in the prediction. On Arria II the measured delay was 31.953 ns.

For the BCLA + Dual2ChUni candidate on Cyclone IV, the predicted worst-case delay was 41.1 ns, driven by the BCLA adder, while the measured delay was 50.296 ns, a difference of 9.2 ns. On Arria II the measured delay was 26.938 ns benefiting from the wide input of the ALUT.

3.5.2 Results

Across all candidates and both FPGA platforms, the worst-case propagation delay was always triggered by an arithmetic instruction rather than a shift or logic instruction, confirming that the arithmetic subsystem sets the ExecUnit critical path at $N=64$. This is consistent with the bottleneck analysis in Section 3.2.2.1. The only exception was ConditionalSum, where the shift was the predicted bottleneck due to its recursive critical path, though the measured result still showed arithmetic as the dominant delay.

Looking at the cost side of the table, CarrySkip + SingleZU2ChRev is the cheapest candidate at 7.67 normalized cost on Cyclone IV and 7.88 on Arria II. However its worst-case delay of 82.682 ns on Cyclone IV is by far the worst of any candidate, producing a cost-performance ratio of 628.94 that makes it uncompetitive despite its area efficiency. The Arria II result of 35.780 ns is far more reasonable, suggesting the Cyclone IV result is a routing artefact rather than a fundamental architectural limitation, but it still ranks last among Arria II candidates at 281.94.

On the opposite end, CarrySelect + SingleZU2ChSep achieves the fastest delay at 31.909 ns on Cyclone IV and 26.115 ns on Arria II, but carries the second highest normalized cost at 11.76 and 12.27 respectively. ConditionalSum + SingleZU2ChSep is the most expensive at 12.78 and 13.61, and despite a competitive 40.939 ns on Cyclone IV its ratio of 523.09 ranks second worst, confirming that its recursive area overhead is never sufficiently offset by timing gains on either platform.

BCLA + Dual2ChUni sits in the middle of the cost range at 9.52 on Cyclone IV but its 50.296 ns delay produces a ratio of 478.86, making it uncompetitive on Cyclone IV. On Arria II however it achieves 26.938 ns with the second fastest behind CarrySelect at a normalized cost of 9.61, yielding the best ratio of 258.89 across all N=64 candidates. This crossover between platforms highlights that Arria II's wider ALUTs absorb the BCLA's wide fan-in logic far more effectively than Cyclone IV.

This leaves a clear gap in the table: no single candidate simultaneously achieves low cost and fast timing on Cyclone IV. The fastest candidate wastes area, the cheapest candidate is slow, and the best ratio candidates are platform-dependent. This motivates exploring whether pairing a performance-optimized arithmetic unit with a cost-optimized shift unit could produce a new candidate that occupies a better position on the cost-performance frontier than any of the existing combinations, which is examined in the following section.

3.6 Design Candidate Considerations

The CarrySelect adder with unequal block sizes of 4, 4, 8, 8, 16, 16, and 8 bits represents the performance-optimized choice for the ArithUnit, as unequal block sizing is specifically tuned to balance ripple delay across blocks and minimize the critical path. Despite this, it incurs a significant area penalty, ranking as the second most resource-intensive topology after the ConditionalSum adder, a characteristic that presents a clear cost-performance trade-off. However, this trade-off can be set aside, as the primary objective for the ArithUnit is strictly performance optimization.

At the other end, the SingleZU2ChRev architecture represents the most resource-efficient ShiftUnit topology. It achieves this by consolidating all functionality into a single 6-stage chain of 2-1 channel muxes merging left and right logical shifts into a single shifter via a reversal chain, and handling both 32-bit and 64-bit instructions within the same chain by clamping the shift amount rather than running a parallel datapath. These design choices

eliminate the redundant hardware present in other topologies, resulting in the lowest resource usage of all ShiftUnit candidates.

Experimental evaluation revealed that the ExecUnit is bottlenecked by arithmetic instructions rather than shift instructions, making it optimal to prioritize performance optimization in the ArithUnit rather than the ShiftUnit. Consequently, the performance-focused CarrySelect adder was combined with the cost-optimized SingleZU2ChRev ShiftUnit. The low resource utilization of the SingleZU2ChRev architecture effectively offsets the higher area penalty of the CarrySelect adder.

Thus the combined design candidate CarrySelect + SingleZU2ChRev will be tested on Cyclone IV and Arria II.

Cyclone IV:

Area = 1349 LEs

Worse Delay = 31.9 ns

Arria II

Area = 610 ALMs

Worse Delay = 26.1 ns

3.7 Optimized Cost-Performance Design Candidate

The best optimized cost-performance candidate(OCPC) is the CarrySelect architecture for the ArithUnit subsystem and the SingleZU2ChRev architecture for the ShiftUnit system on Arria II for N = 64 (highlighted green).

$$\text{Cost} - \text{Performance Ratio} = \text{Normalized Cost} \times \text{Worst Sum Delay(ns)}$$

Thus, the lower the Cost-Performance Ratio, the higher the efficiency.

Candidate (Arith Unit + Shift Unit)	FPGA	Area (measured)	Area (LE equivalent)	Normalized Cost	Worst Propagation Delay	Cost-Performance Ratio
CarrySelect + SingleZU2ChSep	Cyclone IV	1881 LEs	1881 LEs	11.76	31.9 ns	375.14
CarrySelect + SingleZU2ChSep	Arria II	825 ALMs	1963.5 LEs	12.27	26.1 ns	320.37

CarrySkip + SingleZU2ChRev	Cyclone IV	1227 LEs	1227 LEs	7.67	82.7 ns	628.94
CarrySkip + SingleZU2ChRev	Arria II	530 ALMs	1261.4 LEs	7.88	35.8 ns	281.94
BCLA + Dual2ChUni	Cyclone IV	1523 LEs	1523 LEs	9.52	50.3 ns	478.86
BCLA+ Dual2ChUni	Arria II	646 ALMs	1537.48 LEs	9.61	26.9 ns	258.89
ConditionalSum + SingleZU2ChSep	Cyclone IV	2044 LEs	2044 LEs	12.78	40.9 ns	523.09
ConditionalSum + SingleZU2ChSep	Arria II	915 ALMs	2177.7 LEs	13.61	31.9 ns	434.84
CarrySelect + SingleZU2ChRev	Cyclone IV	1349 LEs	1349 LEs	8.37	31.9 ns	267.29
CarrySelect + SingleZU2ChRev	Arria II	610 ALMs	1451.8 LEs	9.02	26.1 ns	235.42

Table 5. ExecUnit Design Candidate Cost and Performance Results at N=64

Candidate (Arith Unit + Shift Unit)	FPGA	Area (measured)	Area (LE equivalent)	Normalized Cost	Worst Propagation Delay	Cost-Performan ce Ratio
CarrySelect + SwapUp2Ch	Cyclone IV	903 LEs	903 LEs	5.64	27.159 ns	153.18
CarrySelect + SwapUp2Ch	Arria II	402 ALMs	956.76 LEs	5.98	18.233 ns	109.01
CarrySkip + Rev2Ch64	Cyclone IV	602 LEs	602 LEs	3.76	26.305 ns	98.888
CarrySkip + Rev2Ch64	Arria II	254 ALMs	604.52 LEs	3.78	34.299 ns	129.65
Block CarryLook-Ahead + DualChainBarrel	Cyclone IV	734 LEs	734 LEs	4.59	35.719 ns	163.95

Block CarryLook-Ahead + DualChainBarrel	Arria II	298 ALMs	709.24 LEs	4.43	17.969 ns	79.56
ConditionalSum + SwapUp2Ch	Cyclone IV	956 LEs	956 LEs	5.98	35.641 ns	213.13
ConditionalSum + SwapUp2Ch	Arria II	417 ALMs	992.46 LEs	6.20	21.348 ns	132.36

Table 6. ExecUnit Design Candidate Cost and Performance Results at N=32

Candidate (Arith Unit + Shift Unit)	FPGA	Area (measured)	Area (LE equivalent)	Normalized Cost	Worst Propagation Delay	Cost-Perfor mance Ratio
CarrySelect + SwapUp2Ch	Cyclone IV	403 LEs	403 LEs	2.52	14.902 ns	37.55
CarrySelect + SwapUp2Ch	Arria II	189 ALMs	449.82 LEs	2.81	22.476 ns	63.16
CarrySkip + Rev2Ch64	Cyclone IV	280 LEs	280 LEs	1.75	18.129 ns	31.73
CarrySkip + Rev2Ch64	Arria II	120 ALMs	285.6 LEs	1.78	16.038 ns	28.55
Block CarryLook-Ahead + DualChainBarrel	Cyclone IV	369 LEs	369 LEs	2.30	15.546 ns	35.76
Block CarryLook-Ahead + DualChainBarrel	Arria II	170 ALMs	404.6 LEs	2.53	14.201 ns	35.92
ConditionalSum + SwapUp2Ch	Cyclone IV	426 LEs	426 LEs	2.66	21.932 ns	58.34
ConditionalSum + SwapUp2Ch	Arria II	190 ALMs	452.2 LEs	2.83	17.382 ns	49.19

Table 7. ExecUnit Design Candidate Cost and Performance Results at N=16

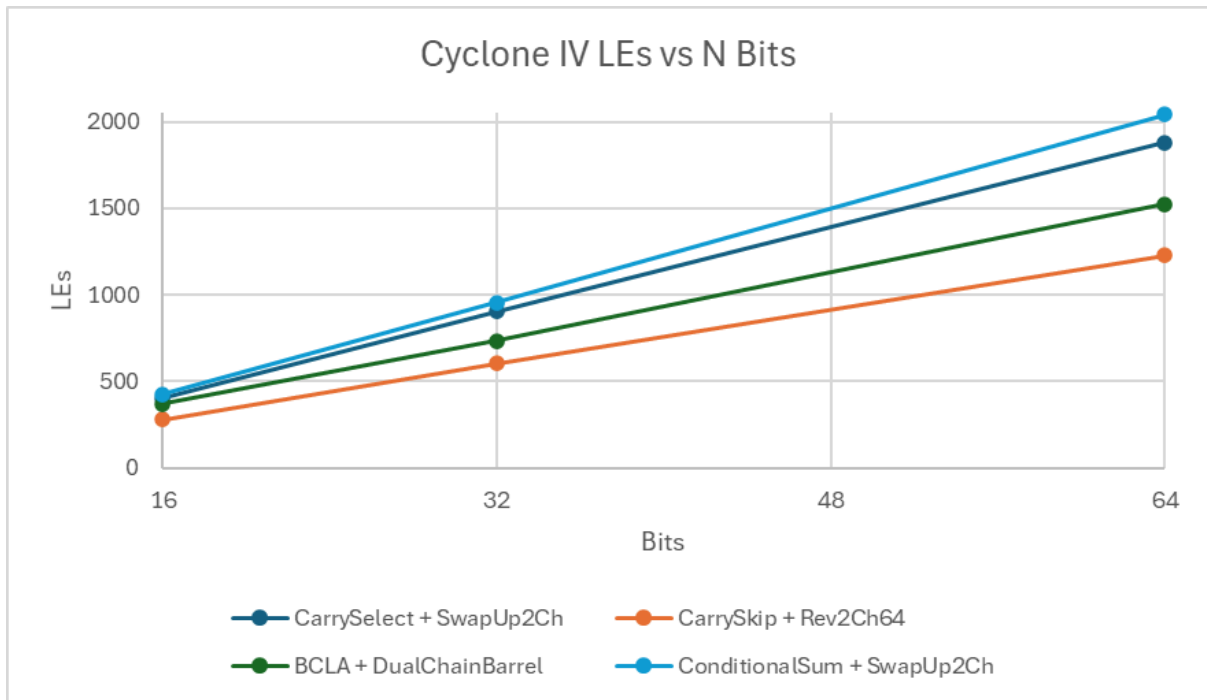


Figure 15. Usage Graph for Cyclone IV (N = 16, 32, 64)

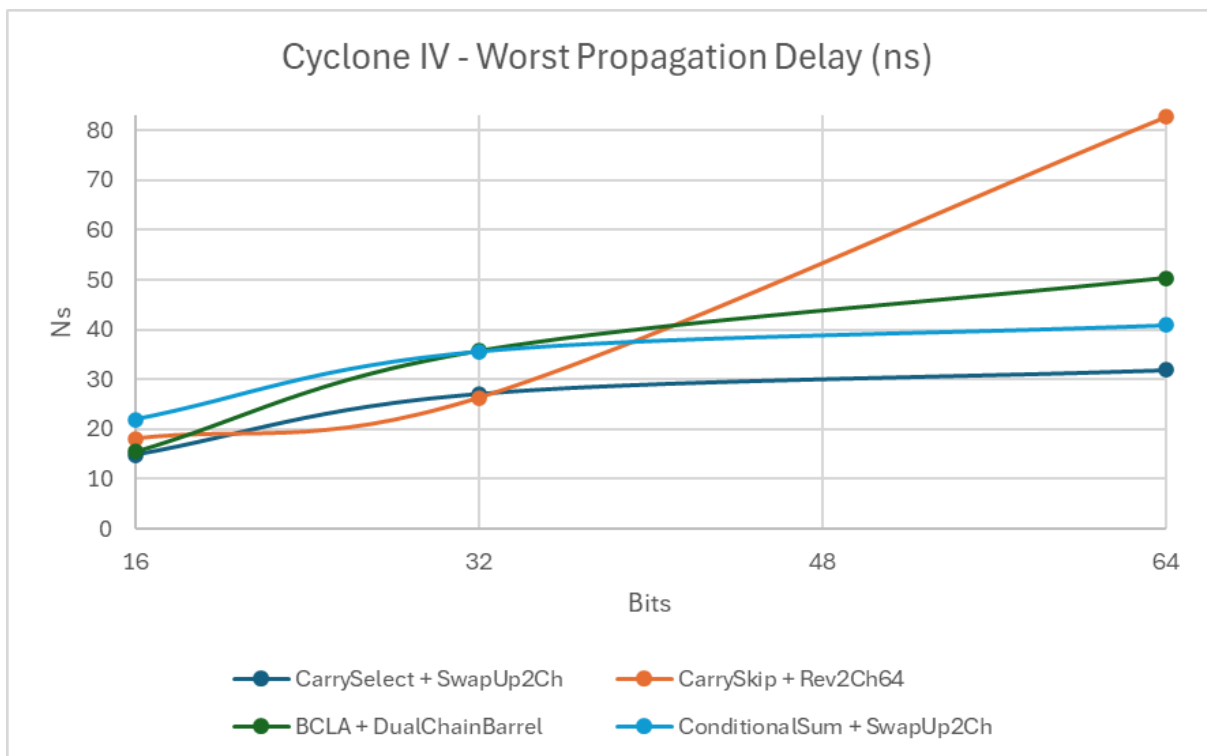


Figure 16. Worst Propagation Delay for Cyclone IV (N = 16, 32, 64)

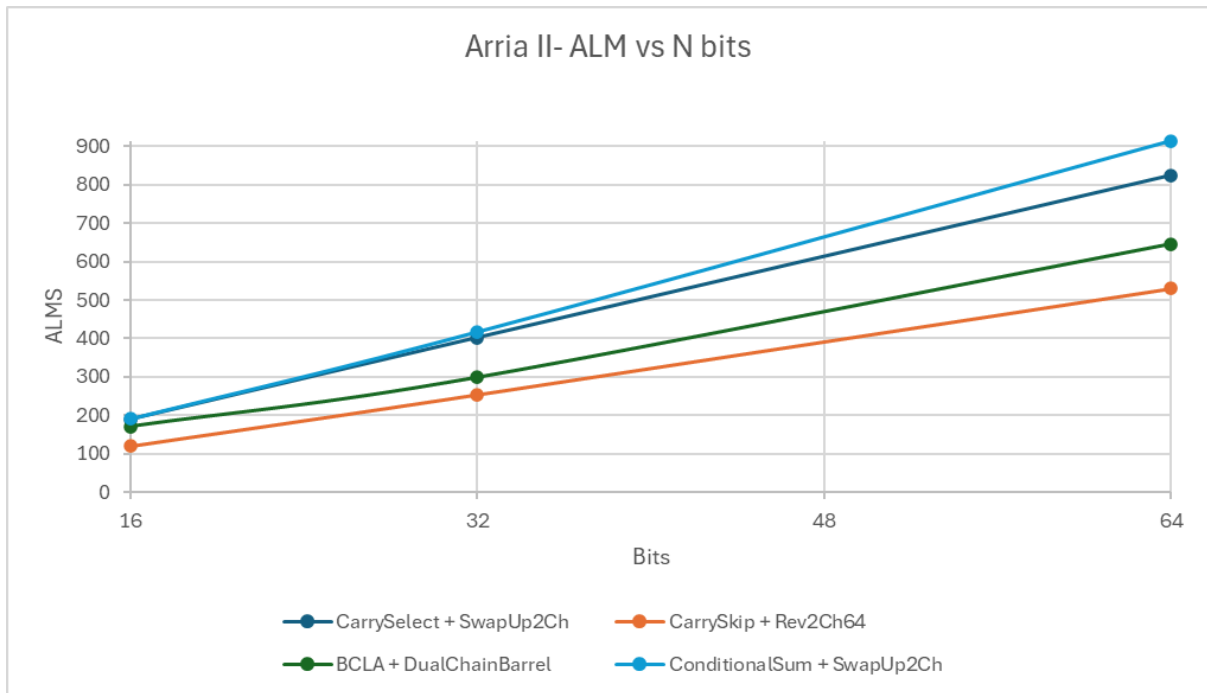


Figure 17. ALMS vs Bits for Arria II (N = 16, 32, 64)

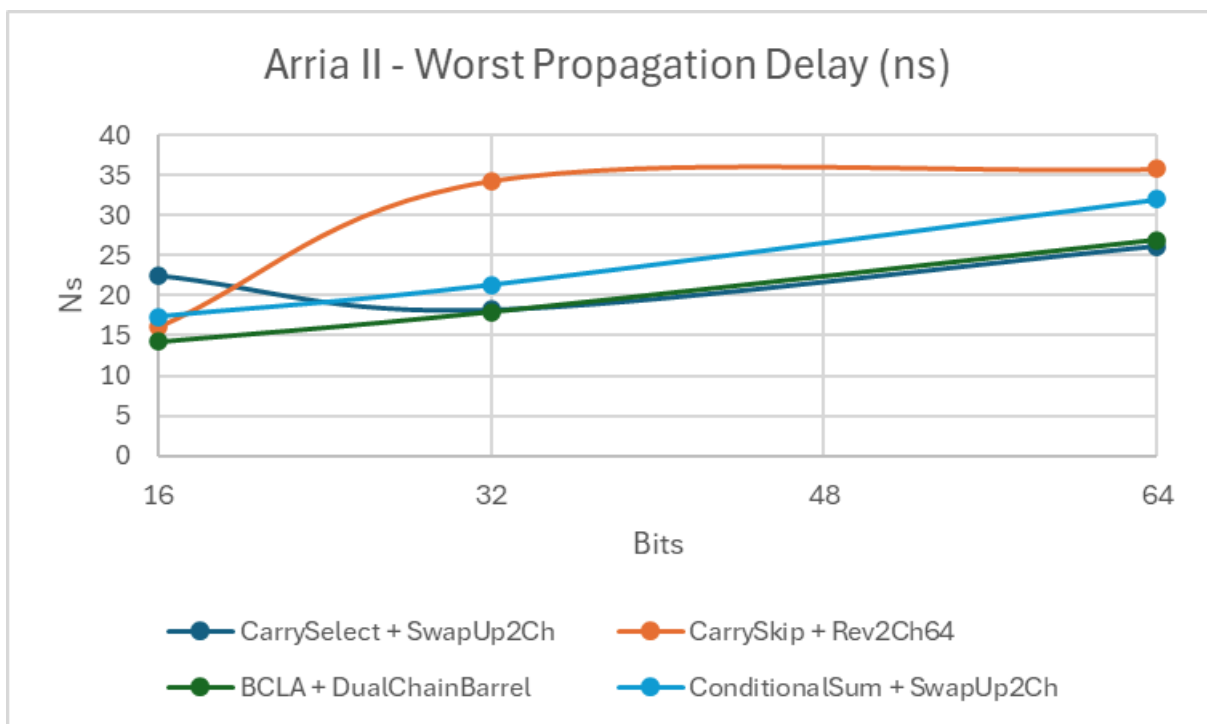


Figure 18. Worst Propagation Delay for Arria II (N = 16, 32, 64)

Analysis of Graphs

Cyclone IV LEs vs N Bits (Figure X)

All four candidates scale approximately linearly with N, converging within 280–426 LEs at N=16 before diverging significantly at N=64, where CarrySkip uses 1227 LEs versus

ConditionalSum + SwapUp2Ch at 2044 LEs. CarrySkip + Rev2Ch64 is the most area-efficient topology across all widths, while ConditionalSum + SwapUp2Ch is consistently the most expensive due to its recursive structure and separate shifter overhead.

Cyclone IV Worst Propagation Delay (Figure X)

CarrySkip + Rev2Ch64 degrades dramatically from 18.129 ns at N=16 to 82.682 ns at N=64, establishing a clear breakpoint below N=32 where it is competitive but above which it becomes the worst candidate by a large margin. CarrySelect + SwapUp2Ch is the speed-optimal topology at larger widths, while at N=16 all candidates converge within 7 ns as the shift subsystem dominates equally.

Arria II ALMs vs N Bits (Figure X)

CarrySkip + Rev2Ch64 remains the most area-efficient topology, and the spread between candidates is more compressed than on Cyclone IV due to Arria II's wider ALUTs absorbing complex logic more efficiently. All candidates converge within 120–190 ALMs at N=16, with the spread widening to 385 ALMs at N=64.

Arria II Worst Propagation Delay (Figure X)

A clear line crossing between CarrySkip + Rev2Ch64 and BCLA + DualChainBarrel establishes a breakpoint near N=24, below which CarrySkip is competitive and above which BCLA dominates as the fastest topology at 17.969 ns and 26.938 ns for N=32 and N=64 respectively. CarrySelect + SwapUp2Ch shows an unusual non-monotonic trend, improving from N=16 to N=32 before increasing at N=64, suggesting its block partitioning is not optimal at very small widths.

Analysis & Optimization

To investigate how operand width affects the cost and performance of each design candidate, all eight ExecUnit configurations were synthesized and simulated at N=64, N=32, and N=16 on both Cyclone IV and Arria II FPGAs. The results are summarized in the tables above.

Effect of Operand Width on Cost

As N decreases from 64 to 16, the normalized cost of all candidates drops significantly. For example, CarrySelect + SwapUp2Ch on Cyclone IV decreases from 11.76 at N=64 to 5.64 at N=32 and 2.52 at N=16. This is expected, as both the arithmetic and shift subsystems scale directly with N — shorter carry chains require fewer LEs, and fewer barrel shifter stages are active for smaller shift amounts. At N=16, all candidates fall below a normalized cost of 3.0, indicating that topological differences become less impactful at smaller widths since the absolute resource count is small regardless of architecture.

On Arria II, the LE-equivalent normalized cost is consistently higher than the corresponding Cyclone IV implementation at N=64, due to the ALM-to-LE conversion factor of 2.38. However this gap narrows at smaller N values, as the arithmetic subsystem, which benefits most from Arria II's wider ALUTs, contributes a smaller fraction of the total resource usage relative to the shift subsystem.

Effect of Operand Width on Performance

Propagation delay improves consistently as N decreases across all candidates. CarrySelect + SwapUp2Ch on Cyclone IV improves from 31.909 ns at N=64 to 27.159 ns at N=32 and 14.902 ns at N=16. The improvement is most pronounced between N=32 and N=16, as at N=16 the arithmetic critical path is short enough that the shift subsystem becomes the dominant bottleneck rather than the carry chain.

Arria II maintains a performance advantage over Cyclone IV at N=64 for all candidates, with the most dramatic difference observed in BCLA + DualChainBarrel, where Arria II achieves 26.938 ns versus 50.296 ns on Cyclone IV — nearly a 2× speedup. This advantage is attributed to Arria II's superior interconnect architecture and wider ALUTs, which better absorb the wide fan-in logic of the block-level generate and propagate terms. However, this advantage diminishes at smaller N. At N=16, CarrySkip + Rev2Ch64 on Cyclone IV achieves 18.129 ns versus 16.038 ns on Arria II, a gap of only 2.1 ns compared to the much larger differences at N=64. This confirms the architecture-dependent performance crossover observed in prior subsystem-level experiments — Cyclone IV's raw gate speed becomes more competitive when carry chains are short.

Identifying the Optimal Topology per N

The cost-performance ratio, defined as Normalized Cost × Worst Propagation Delay (ns), reveals that the optimal topology is N-dependent. At N=64, BCLA + DualChainBarrel on Arria II achieves the best ratio of 258.89, benefiting from Arria II's interconnect advantage on the wide look-ahead logic. At N=32, BCLA + DualChainBarrel on Arria II again leads with a ratio of 79.56, a dramatic improvement over N=64 that reflects the sharper timing gains on Arria II as the carry chain shortens. At N=16, however, the optimal candidate shifts to CarrySkip + Rev2Ch64 on Arria II with a ratio of 28.55, as the carry-skip architecture becomes more effective at shorter chain lengths where its skip logic can more frequently bypass blocks, while Rev2Ch64's compact single-chain reversal structure keeps resource usage low.

Conversely, CarrySkip + Rev2Ch64 on Cyclone IV exhibits the worst propagation delay at N=64 (82.682 ns), suggesting that the skip logic does not interact efficiently with Cyclone IV's carry chain primitives at full 64-bit width. This anomaly largely disappears at smaller N, where CarrySkip becomes competitive and even optimal at N=16 on Arria II.

References

<https://www.digikey.ca/en/products/detail/intel/EP2AGX45DF29C6G/5429581>

<https://www.digikey.ca/en/products/detail/altera/EP4CE115F29C7/2260452>

<https://cdrdv2-public.intel.com/653974/cyclone4-handbook.pdf>

https://cdrdv2-public.intel.com/654011/arria-ii-gx_handbook.pdf

Appendix

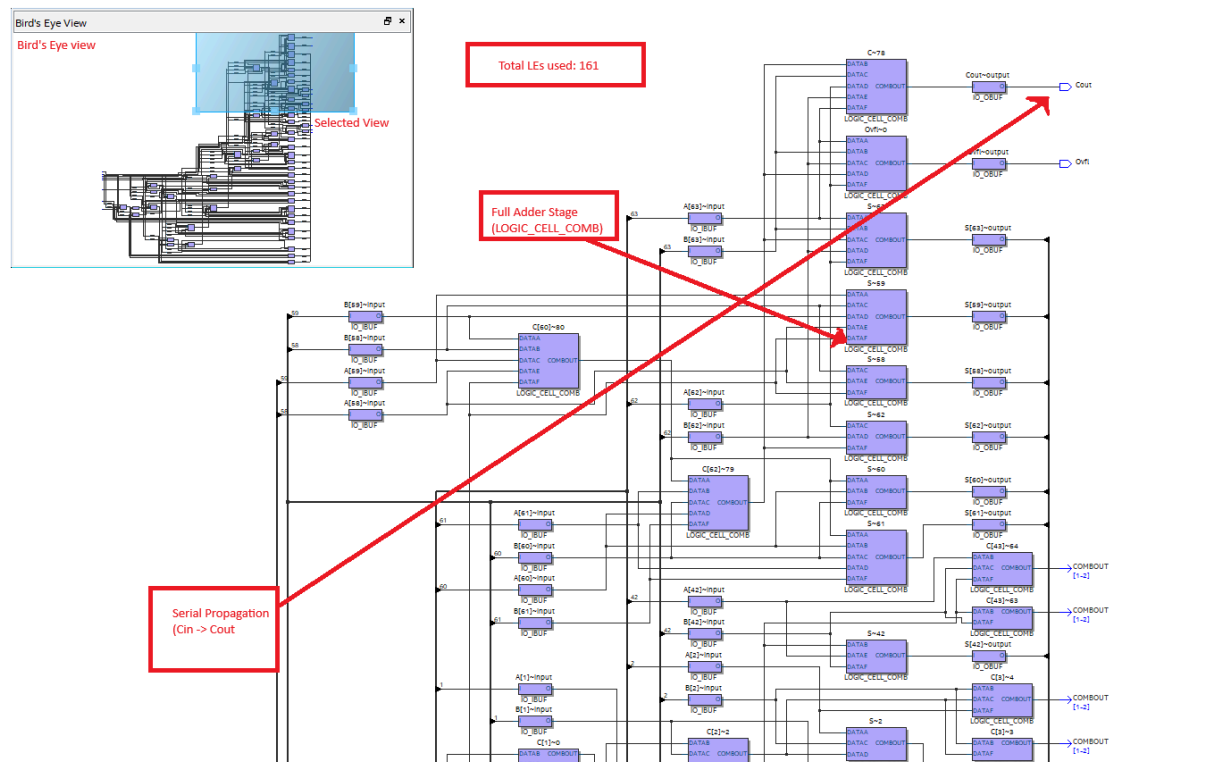


Figure A.1: Post-fit Technology Map of the baseline ripple adder (Cyclone IV). Each `LOGIC_CELL_COMB` forms a full-adder stage with carry propagating serially from `CIN` to `COUT`.

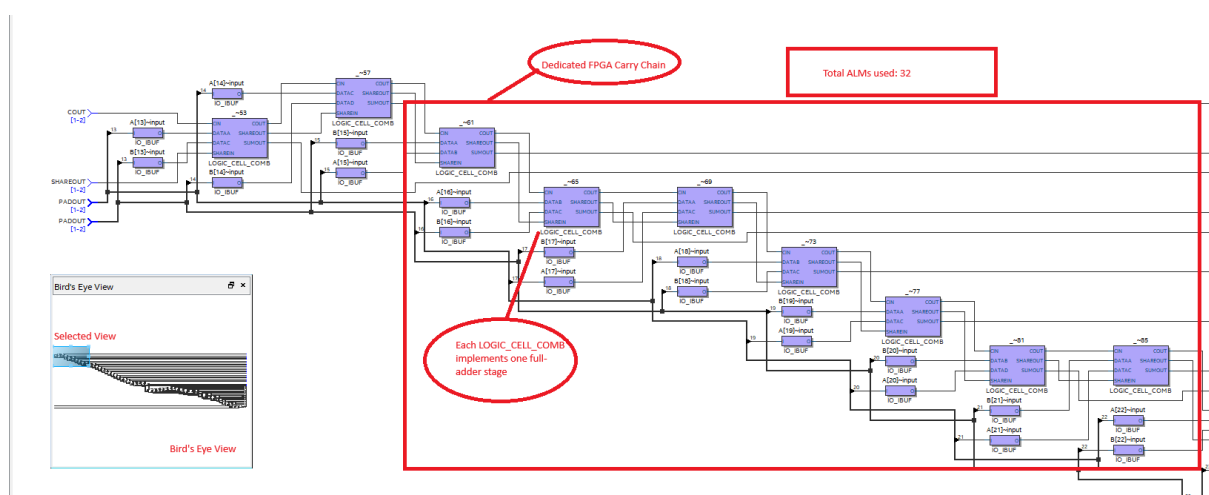


Figure A.2: Post-fit Technology Map of the FastRipple adder (Arria II) showing the FPGA's dedicated carry chain connecting each full-adder stage.

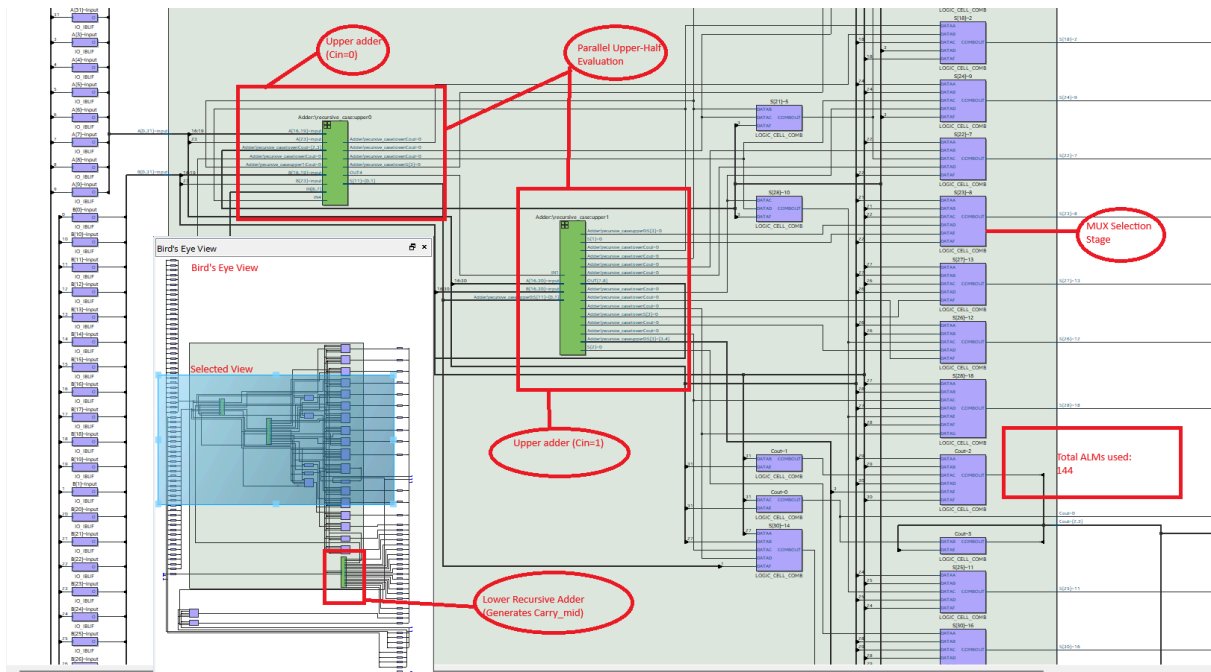


Figure A.3. Post-fit Technology Map of the Conditional Sum Adder (Arria II). The lower ripple block generates carry_mid, which selects between two precomputed upper-half results.

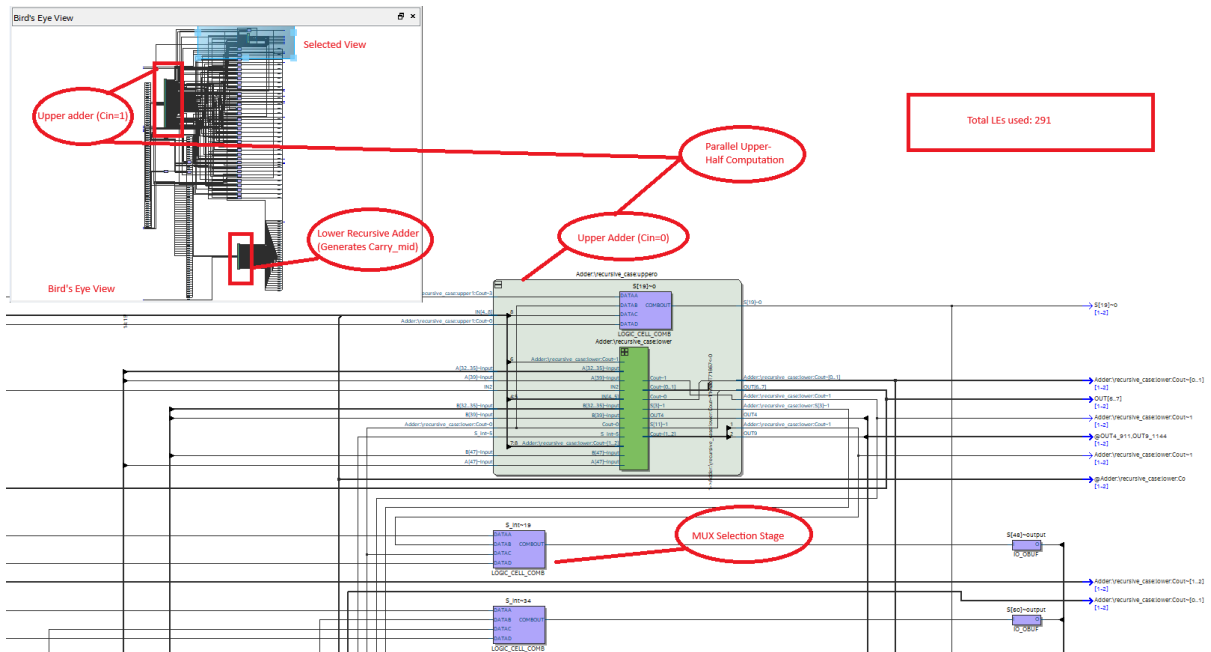


Figure A.4: Post-fit Technology Map of the recursive Conditional Sum Adder (Cyclone IV). The lower block generates carry_mid, which selects between parallel upper-half results through a multiplexer.

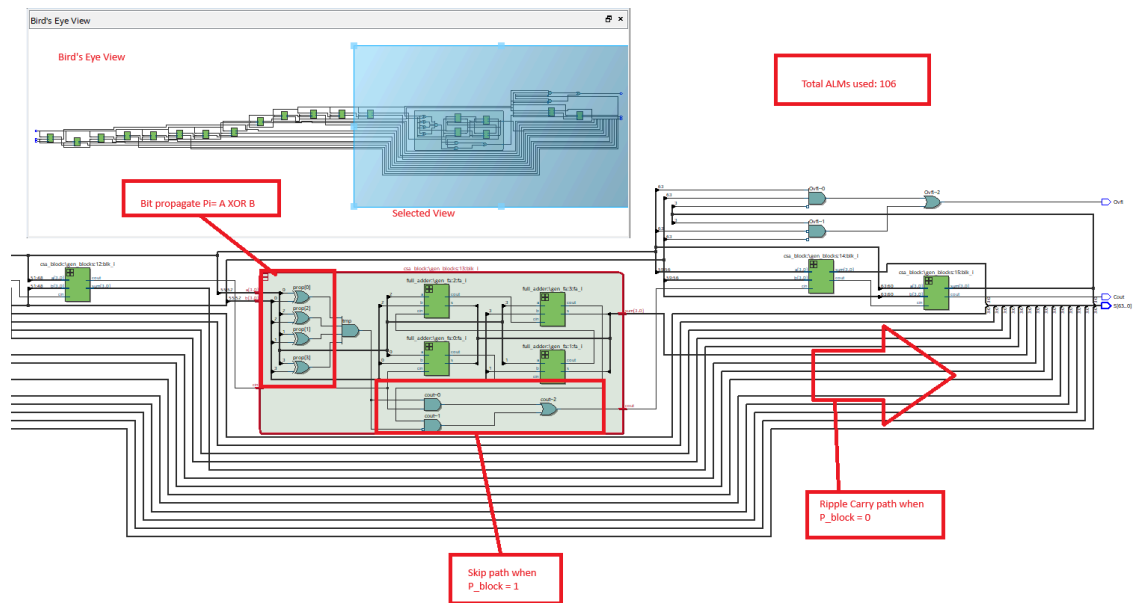


Figure A.5: Post-fit Technology Map of the Carry Skip Adder (Arria II). Each block consists of a ripple-carry chain with additional skip logic, where block propagate signals control a multiplexer that allows the carry to bypass the block when all propagate conditions are satisfied.

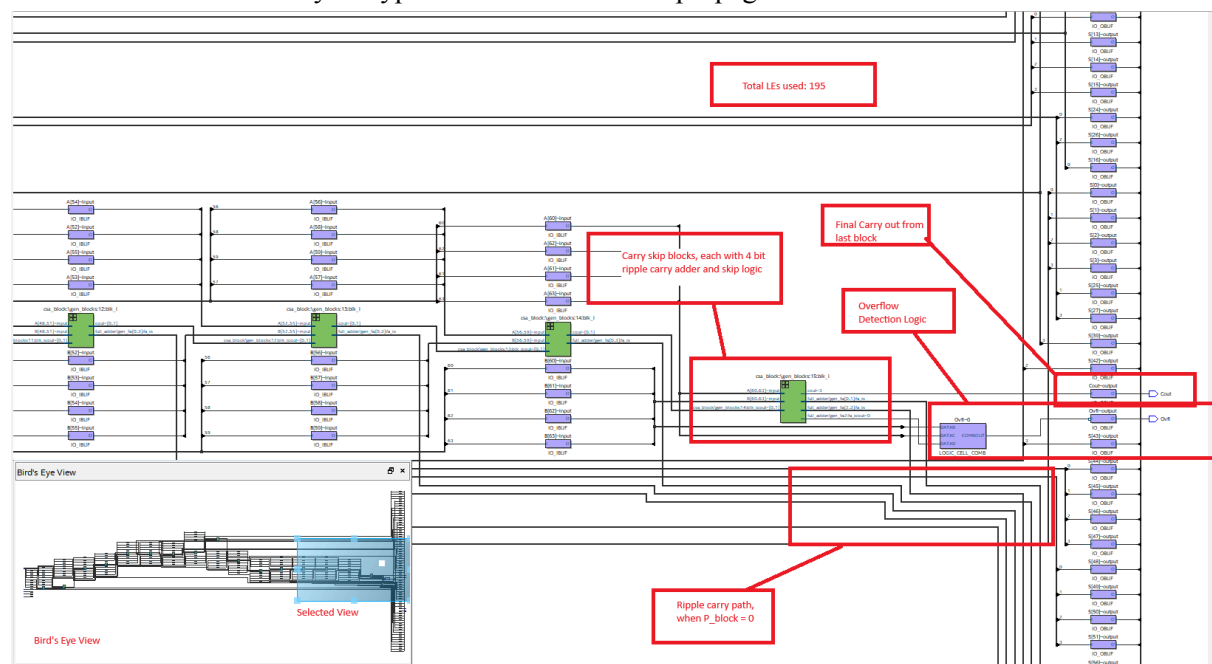


Figure A.6: Post-fit Technology Map of the Carry Skip Adder (Cyclone IV). Each block consists of a ripple-carry chain with additional skip logic, where block propagate signals control a multiplexer that allows the carry to bypass the block when all propagate conditions are satisfied.

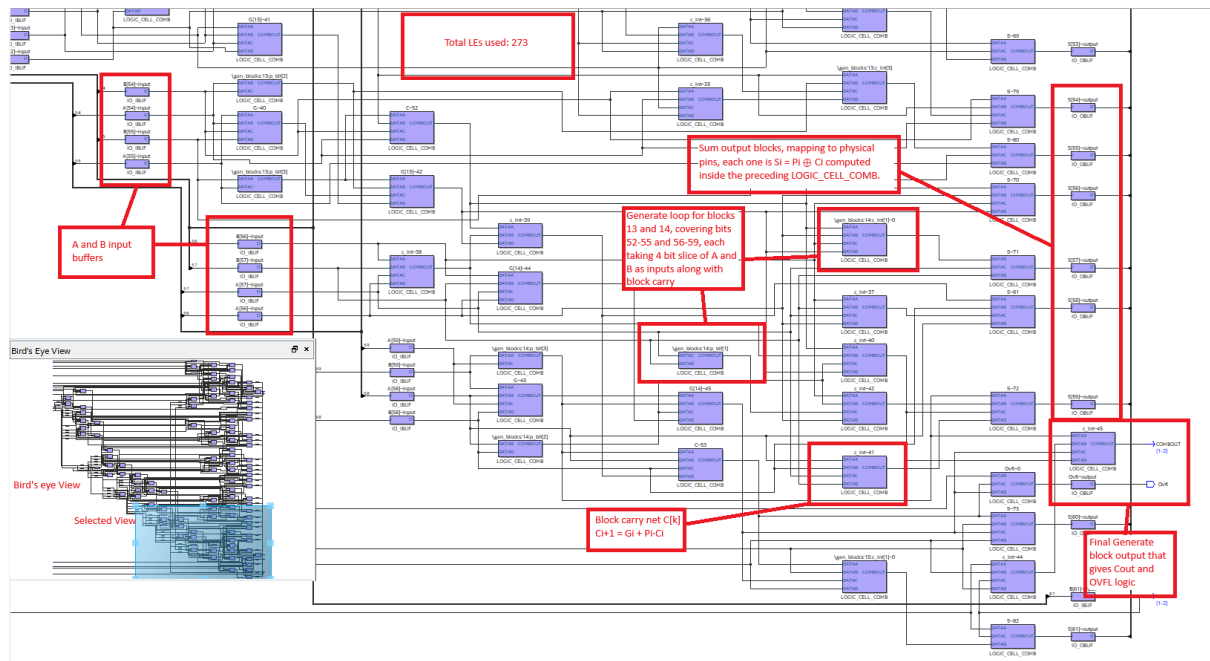


Figure A.7: Post-Fit Technology Map of Block Carry Look-Ahead Adder (Cyclone IV). 16×4 -bit Blocks, Bits 52–63 Region Shown.

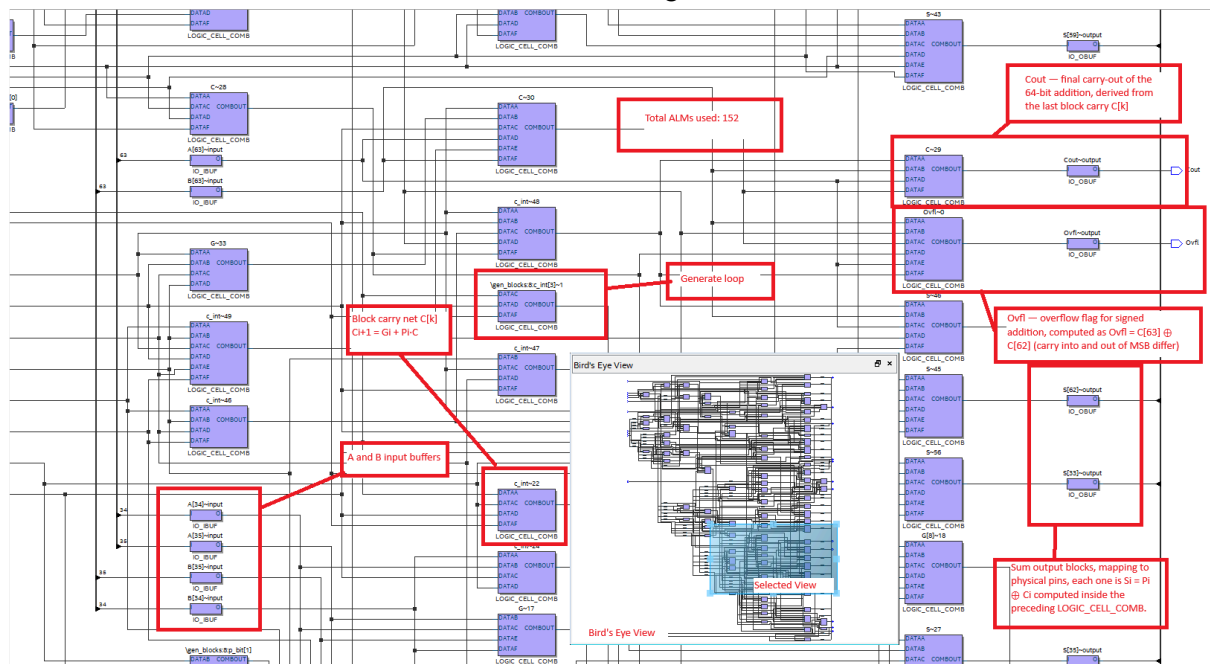


Figure A.8: Post-fit Technology Map of the Block Carry Look-Ahead Adder (Arria II).

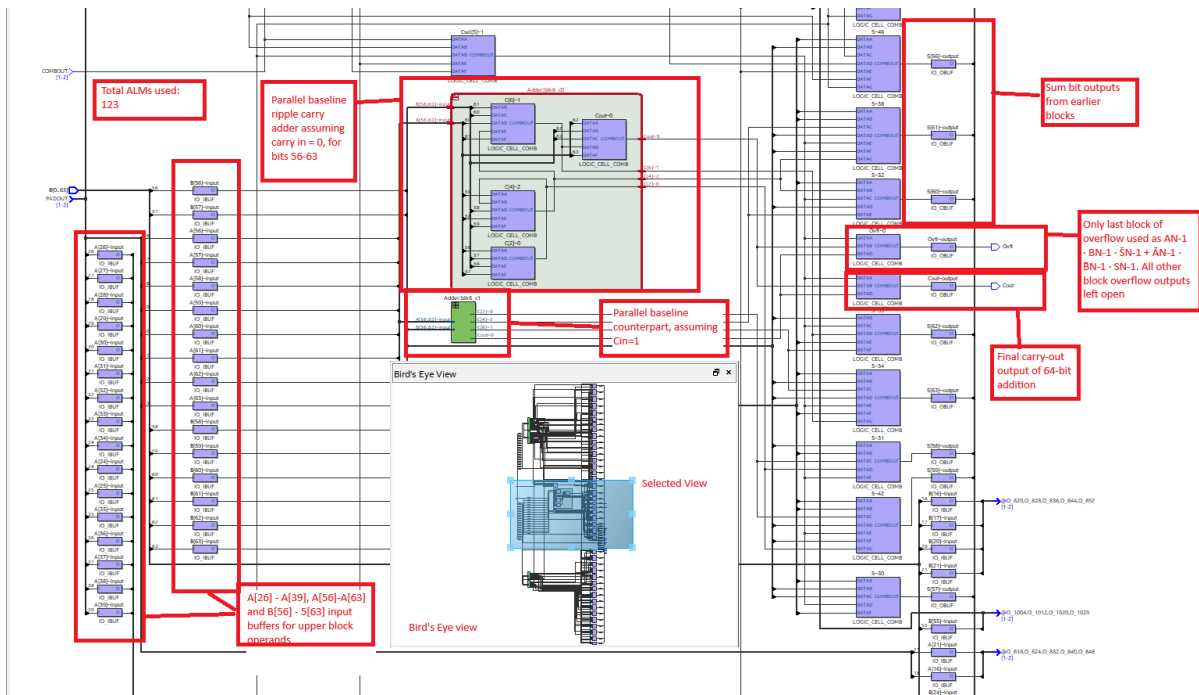


Figure A.9: Post-Fit Technology Map of Carry Select Adder (Arria II). Showing Parallel Cin=0/Cin=1 Baseline Instances, MUX Selection Logic, and Final Overflow/Cout Outputs

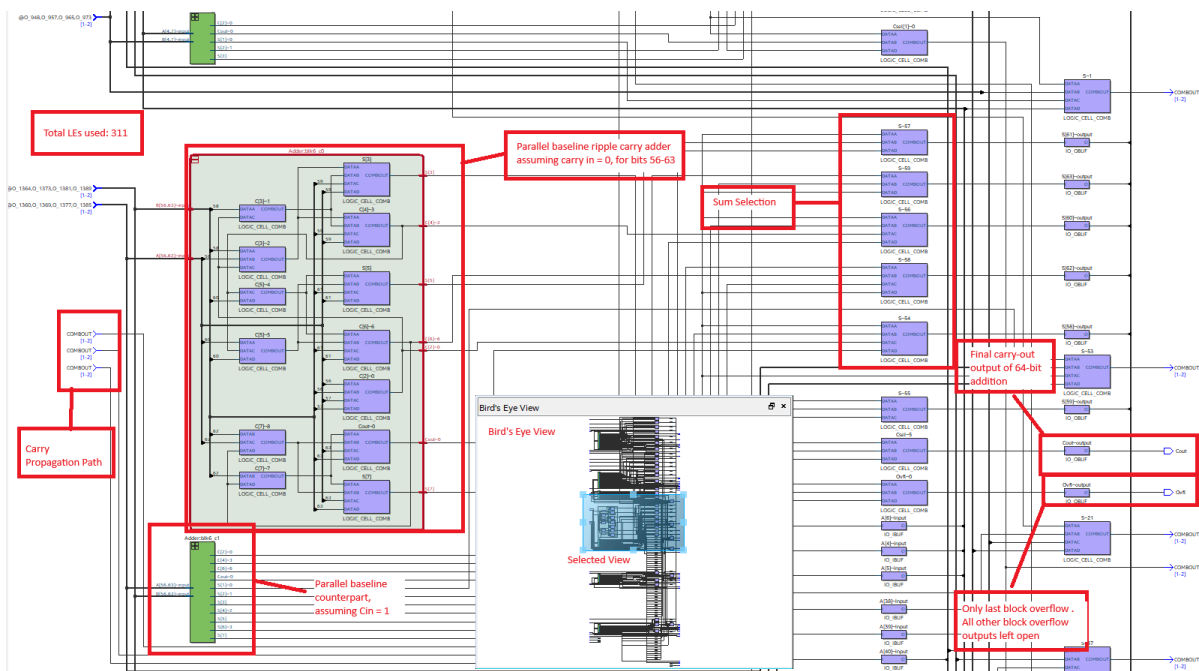


Figure A.10: Post-Fit Technology Map of Carry Select Adder (Cyclone IV). Showing Parallel Cin=0/Cin=1 Baseline Instances, MUX Selection Logic, and Final Overflow/Cout Outputs